

```
In [1]: import pandas as pd
```

```
In [2]: movies = pd.read_csv(r"C:\Users\LIKHITH\OneDrive\Documents\Movie-Rating.csv")
```

```
In [3]: movies
```

```
Out[3]:
```

|            | Film                 | Genre     | Rotten<br>Tomatoes<br>Ratings % | Audience<br>Ratings % | Budget<br>(million \$) | Year of<br>release |
|------------|----------------------|-----------|---------------------------------|-----------------------|------------------------|--------------------|
| <b>0</b>   | (500) Days of Summer | Comedy    | 87                              | 81                    | 8                      | 2009               |
| <b>1</b>   | 10,000 B.C.          | Adventure | 9                               | 44                    | 105                    | 2008               |
| <b>2</b>   | 12 Rounds            | Action    | 30                              | 52                    | 20                     | 2009               |
| <b>3</b>   | 127 Hours            | Adventure | 93                              | 84                    | 18                     | 2010               |
| <b>4</b>   | 17 Again             | Comedy    | 55                              | 70                    | 20                     | 2009               |
| ...        | ...                  | ...       | ...                             | ...                   | ...                    | ...                |
| <b>554</b> | Your Highness        | Comedy    | 26                              | 36                    | 50                     | 2011               |
| <b>555</b> | Youth in Revolt      | Comedy    | 68                              | 52                    | 18                     | 2009               |
| <b>556</b> | Zodiac               | Thriller  | 89                              | 73                    | 65                     | 2007               |
| <b>557</b> | Zombieland           | Action    | 90                              | 87                    | 24                     | 2009               |
| <b>558</b> | Zookeeper            | Comedy    | 14                              | 42                    | 80                     | 2011               |

559 rows × 6 columns

```
In [4]: id(movies)
```

```
Out[4]: 1567995134704
```

```
In [5]: type(movies)
```

```
Out[5]: pandas.core.frame.DataFrame
```

```
In [6]: movies
```

Out[6]:

|            | Film                 | Genre     | Rotten Tomatoes Ratings % | Audience Ratings % | Budget (million \$) | Year of release |
|------------|----------------------|-----------|---------------------------|--------------------|---------------------|-----------------|
| <b>0</b>   | (500) Days of Summer | Comedy    | 87                        | 81                 | 8                   | 2009            |
| <b>1</b>   | 10,000 B.C.          | Adventure | 9                         | 44                 | 105                 | 2008            |
| <b>2</b>   | 12 Rounds            | Action    | 30                        | 52                 | 20                  | 2009            |
| <b>3</b>   | 127 Hours            | Adventure | 93                        | 84                 | 18                  | 2010            |
| <b>4</b>   | 17 Again             | Comedy    | 55                        | 70                 | 20                  | 2009            |
| <b>...</b> | ...                  | ...       | ...                       | ...                | ...                 | ...             |
| <b>554</b> | Your Highness        | Comedy    | 26                        | 36                 | 50                  | 2011            |
| <b>555</b> | Youth in Revolt      | Comedy    | 68                        | 52                 | 18                  | 2009            |
| <b>556</b> | Zodiac               | Thriller  | 89                        | 73                 | 65                  | 2007            |
| <b>557</b> | Zombieland           | Action    | 90                        | 87                 | 24                  | 2009            |
| <b>558</b> | Zookeeper            | Comedy    | 14                        | 42                 | 80                  | 2011            |

559 rows × 6 columns

In [7]: `len(movies)`

Out[7]: 559

In [8]: `import numpy`  
`print(numpy.__version__)`

2.1.3

In [9]: `import pandas`  
`print(pandas.__version__)`

2.2.3

In [10]: `movies.columns`

Out[10]: Index(['Film', 'Genre', 'Rotten Tomatoes Ratings %', 'Audience Ratings %', 'Budget (million \$)', 'Year of release'], dtype='object')

In [11]: `movies.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Film                                559 non-null    object
1   Genre                              559 non-null    object
2   Rotten Tomatoes Ratings %          559 non-null    int64
3   Audience Ratings %                 559 non-null    int64
4   Budget (million $)                 559 non-null    int64
5   Year of release                    559 non-null    int64
dtypes: int64(4), object(2)
memory usage: 26.3+ KB
```

In [12]: `movies.shape`

Out[12]: (559, 6)

In [13]: `movies.head()`

Out[13]:

|          | Film                 | Genre     | Rotten Tomatoes<br>Ratings % | Audience<br>Ratings % | Budget<br>(million \$) | Year of<br>release |
|----------|----------------------|-----------|------------------------------|-----------------------|------------------------|--------------------|
| <b>0</b> | (500) Days of Summer | Comedy    | 87                           | 81                    | 8                      | 2009               |
| <b>1</b> | 10,000 B.C.          | Adventure | 9                            | 44                    | 105                    | 2008               |
| <b>2</b> | 12 Rounds            | Action    | 30                           | 52                    | 20                     | 2009               |
| <b>3</b> | 127 Hours            | Adventure | 93                           | 84                    | 18                     | 2010               |
| <b>4</b> | 17 Again             | Comedy    | 55                           | 70                    | 20                     | 2009               |

In [14]: `movies.tail`

```
Out[14]: <bound method NDFrame.tail of
es Ratings % \
0 (500) Days of Summer Comedy 87
1 10,000 B.C. Adventure 9
2 12 Rounds Action 30
3 127 Hours Adventure 93
4 17 Again Comedy 55
.. ...
554 Your Highness Comedy 26
555 Youth in Revolt Comedy 68
556 Zodiac Thriller 89
557 Zombieland Action 90
558 Zookeeper Comedy 14
```

```

Audience Ratings % Budget (million $) Year of release
0 81 8 2009
1 44 105 2008
2 52 20 2009
3 84 18 2010
4 70 20 2009
.. ...
554 36 50 2011
555 52 18 2009
556 73 65 2007
557 87 24 2009
558 42 80 2011
```

[559 rows x 6 columns]>

```
In [15]: movies.columns = ['Film', 'Genre', 'CriticRating', 'AudienceRating', 'BudgetMillions']
```

```
In [16]: movies.head()
```

```
Out[16]:
```

|   | Film                 | Genre     | CriticRating | AudienceRating | BudgetMillions | Year |
|---|----------------------|-----------|--------------|----------------|----------------|------|
| 0 | (500) Days of Summer | Comedy    | 87           | 81             | 8              | 2009 |
| 1 | 10,000 B.C.          | Adventure | 9            | 44             | 105            | 2008 |
| 2 | 12 Rounds            | Action    | 30           | 52             | 20             | 2009 |
| 3 | 127 Hours            | Adventure | 93           | 84             | 18             | 2010 |
| 4 | 17 Again             | Comedy    | 55           | 70             | 20             | 2009 |

```
In [17]: movies.head(1)
```

```
Out[17]:
```

|   | Film                 | Genre  | CriticRating | AudienceRating | BudgetMillions | Year |
|---|----------------------|--------|--------------|----------------|----------------|------|
| 0 | (500) Days of Summer | Comedy | 87           | 81             | 8              | 2009 |

```
In [18]: movies.describe()
```

Out[18]:

|              | CriticRating | AudienceRating | BudgetMillions | Year        |
|--------------|--------------|----------------|----------------|-------------|
| <b>count</b> | 559.000000   | 559.000000     | 559.000000     | 559.000000  |
| <b>mean</b>  | 47.309481    | 58.744186      | 50.236136      | 2009.152057 |
| <b>std</b>   | 26.413091    | 16.826887      | 48.731817      | 1.362632    |
| <b>min</b>   | 0.000000     | 0.000000       | 0.000000       | 2007.000000 |
| <b>25%</b>   | 25.000000    | 47.000000      | 20.000000      | 2008.000000 |
| <b>50%</b>   | 46.000000    | 58.000000      | 35.000000      | 2009.000000 |
| <b>75%</b>   | 70.000000    | 72.000000      | 65.000000      | 2010.000000 |
| <b>max</b>   | 97.000000    | 96.000000      | 300.000000     | 2011.000000 |

In [19]: `movies.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Film            559 non-null   object
1   Genre           559 non-null   object
2   CriticRating    559 non-null   int64
3   AudienceRating  559 non-null   int64
4   BudgetMillions  559 non-null   int64
5   Year            559 non-null   int64
dtypes: int64(4), object(2)
memory usage: 26.3+ KB
```

In [20]: `movies.describe().transpose()`

Out[20]:

|                       | count | mean        | std       | min    | 25%    | 50%    | 75%    | max    |
|-----------------------|-------|-------------|-----------|--------|--------|--------|--------|--------|
| <b>CriticRating</b>   | 559.0 | 47.309481   | 26.413091 | 0.0    | 25.0   | 46.0   | 70.0   | 97.0   |
| <b>AudienceRating</b> | 559.0 | 58.744186   | 16.826887 | 0.0    | 47.0   | 58.0   | 72.0   | 96.0   |
| <b>BudgetMillions</b> | 559.0 | 50.236136   | 48.731817 | 0.0    | 20.0   | 35.0   | 65.0   | 300.0  |
| <b>Year</b>           | 559.0 | 2009.152057 | 1.362632  | 2007.0 | 2008.0 | 2009.0 | 2010.0 | 2011.0 |

In [21]: `movies.Film = movies.Film.astype('category')`In [22]: `movies.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Film            559 non-null   category
1   Genre           559 non-null   object
2   CriticRating    559 non-null   int64
3   AudienceRating  559 non-null   int64
4   BudgetMillions  559 non-null   int64
5   Year            559 non-null   int64
dtypes: category(1), int64(4), object(1)
memory usage: 43.6+ KB
```

```
In [23]: movies.Genre = movies.Genre.astype('category')
         movies.Year = movies.Year.astype('category')
```

```
In [24]: movies.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Film            559 non-null   category
1   Genre           559 non-null   category
2   CriticRating    559 non-null   int64
3   AudienceRating  559 non-null   int64
4   BudgetMillions  559 non-null   int64
5   Year            559 non-null   category
dtypes: category(3), int64(3)
memory usage: 36.5 KB
```

```
In [25]: movies.Genre
```

```
Out[25]: 0      Comedy
         1      Adventure
         2      Action
         3      Adventure
         4      Comedy
         ...
        554     Comedy
        555     Comedy
        556     Thriller
        557     Action
        558     Comedy
        Name: Genre, Length: 559, dtype: category
        Categories (7, object): ['Action', 'Adventure', 'Comedy', 'Drama', 'Horror', 'Romance', 'Thriller']
```

```
In [26]: movies.describe()
```

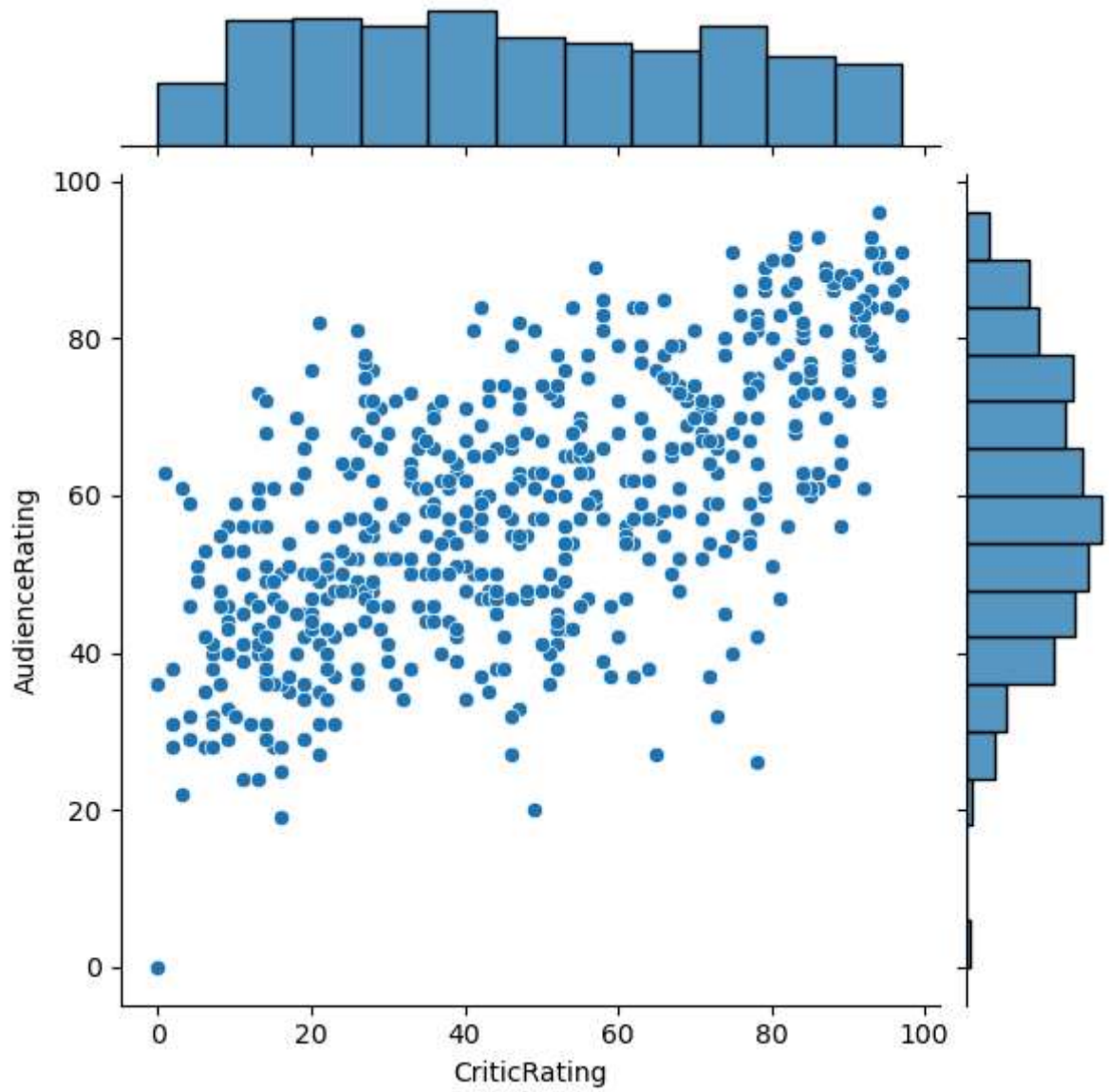
Out[26]:

|       | CriticRating | AudienceRating | BudgetMillions |
|-------|--------------|----------------|----------------|
| count | 559.000000   | 559.000000     | 559.000000     |
| mean  | 47.309481    | 58.744186      | 50.236136      |
| std   | 26.413091    | 16.826887      | 48.731817      |
| min   | 0.000000     | 0.000000       | 0.000000       |
| 25%   | 25.000000    | 47.000000      | 20.000000      |
| 50%   | 46.000000    | 58.000000      | 35.000000      |
| 75%   | 70.000000    | 72.000000      | 65.000000      |
| max   | 97.000000    | 96.000000      | 300.000000     |

```
In [27]: from matplotlib import pyplot as plt
import seaborn as sns

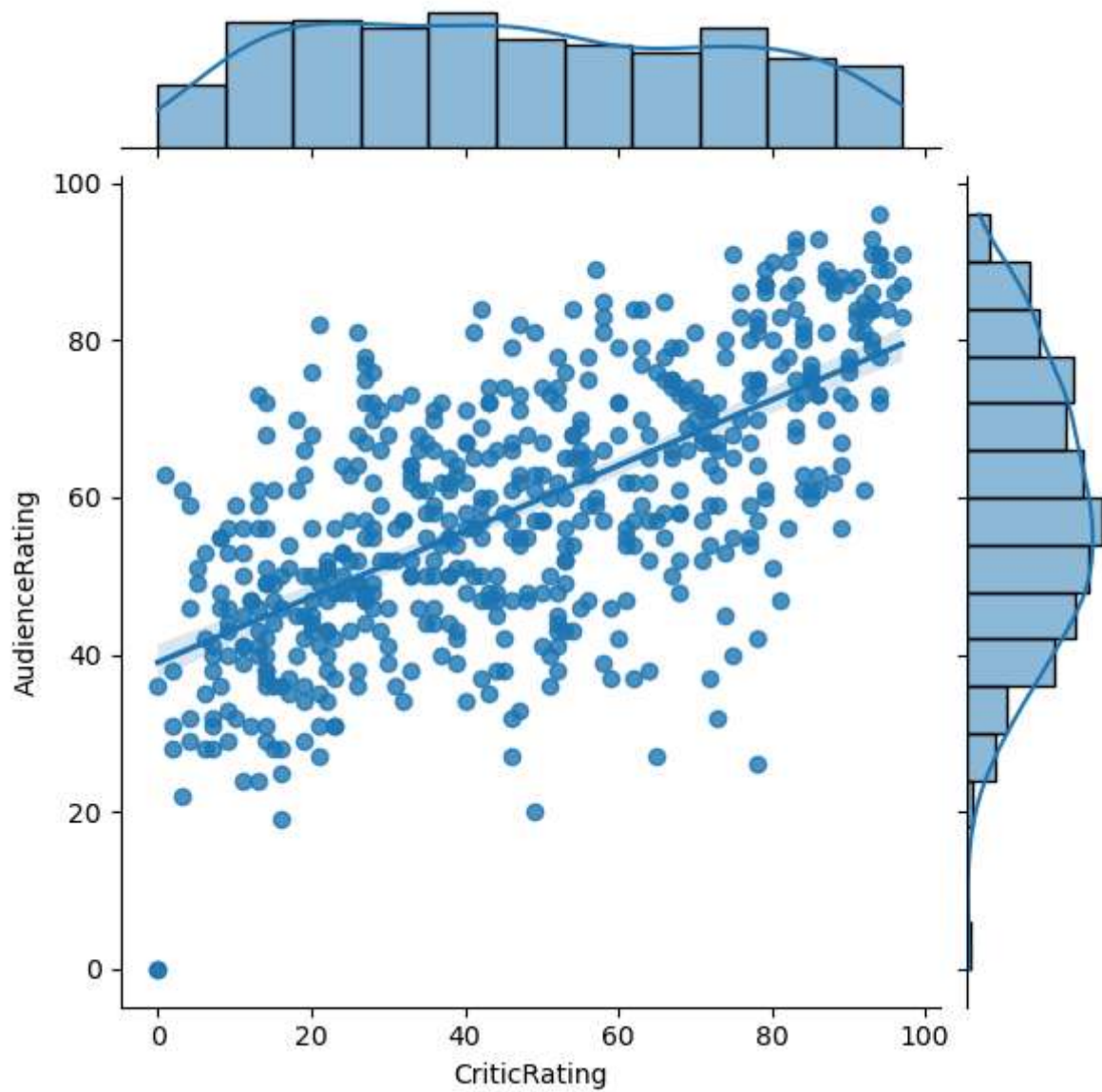
import warnings
warnings.filterwarnings('ignore')
```

```
In [28]: j=sns.jointplot(data=movies, x='CriticRating', y='AudienceRating')
```

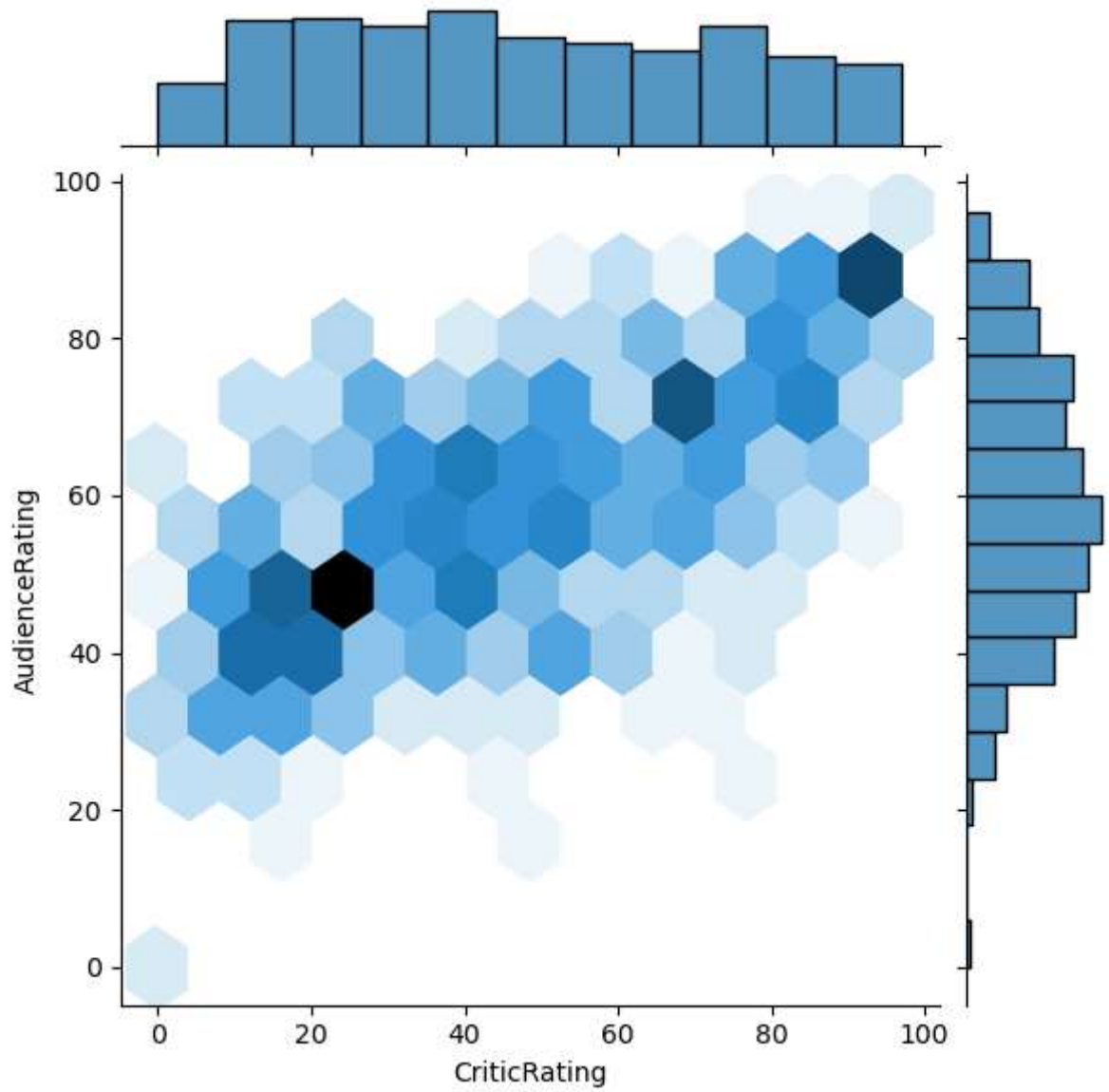


```
In [29]: j=sns.jointplot(data=movies, x='CriticRating', y='AudienceRating', kind='reg')
```

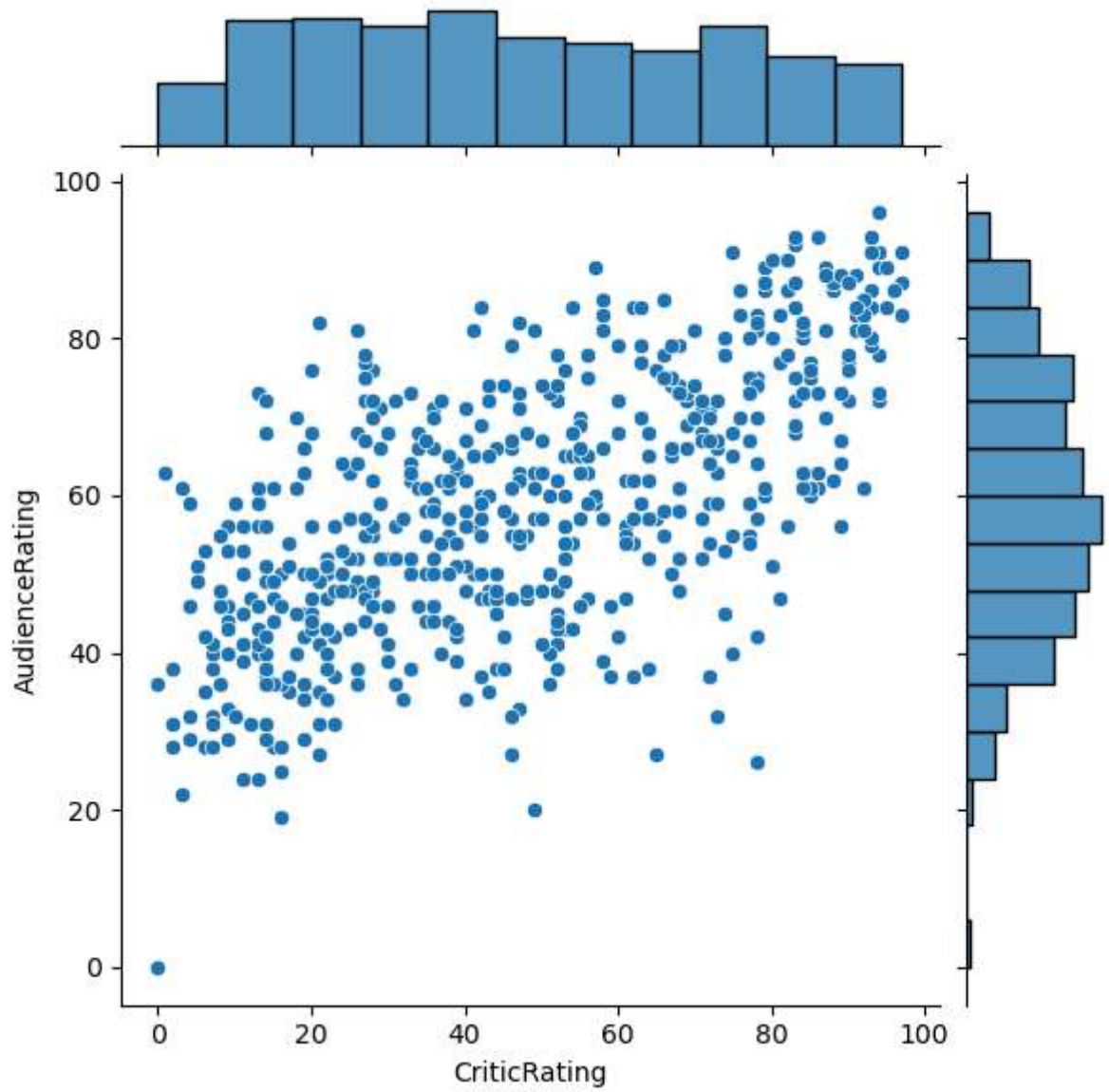




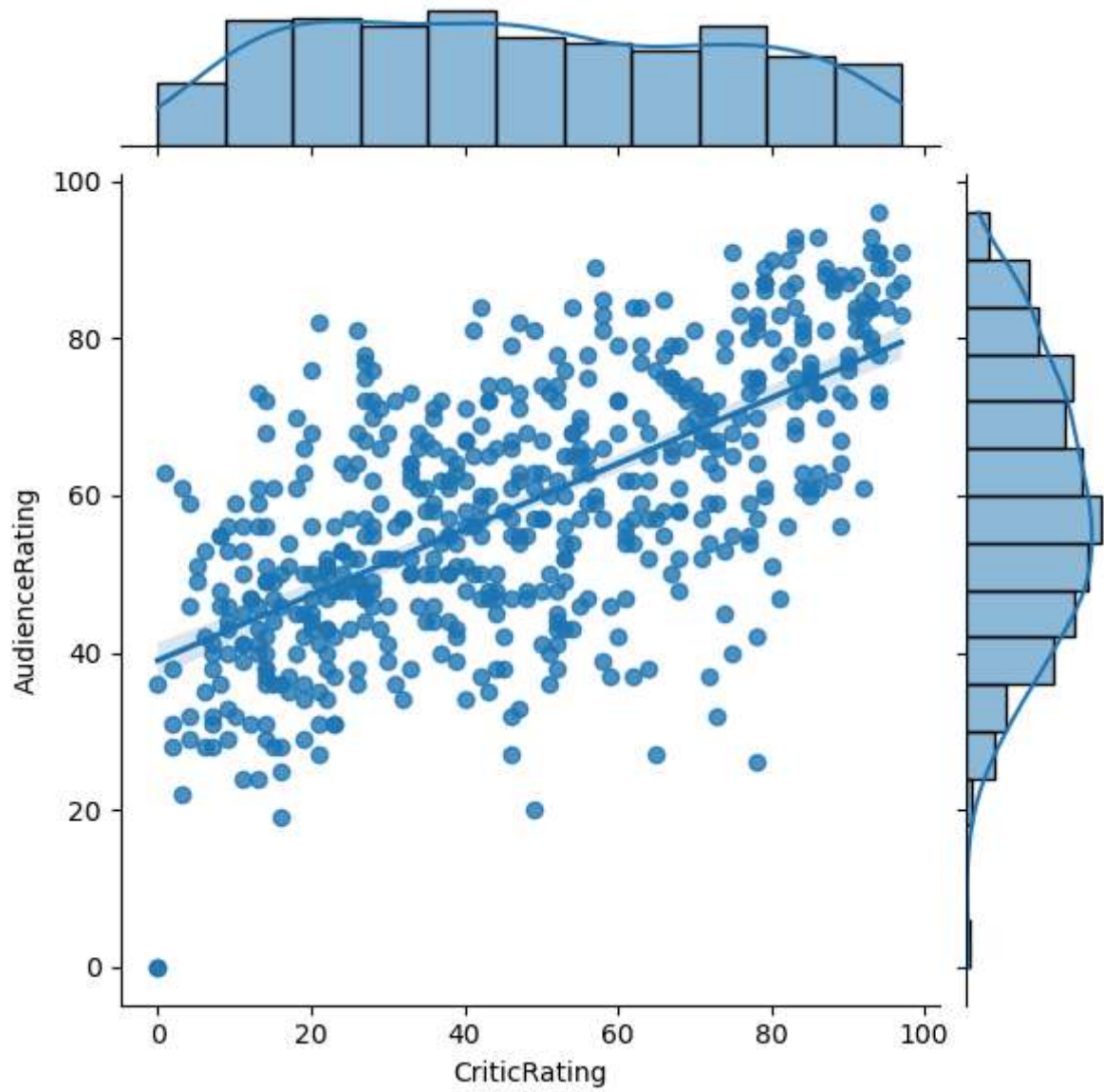
```
In [30]: j=sns.jointplot(data=movies, x='CriticRating', y='AudienceRating', kind='hex')
```



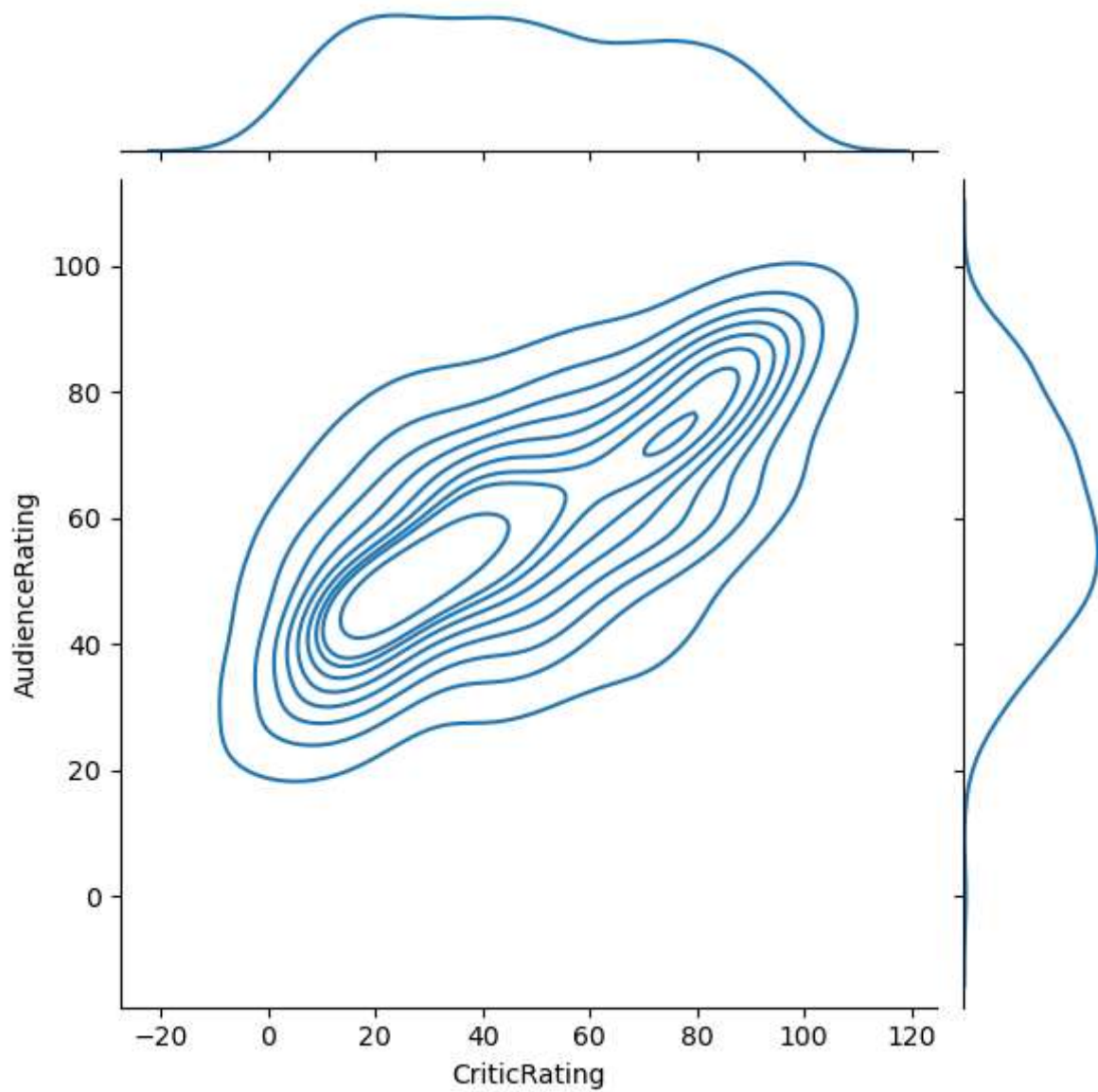
```
In [31]: j=sns.jointplot(data=movies, x='CriticRating', y='AudienceRating', kind='scatter')
```



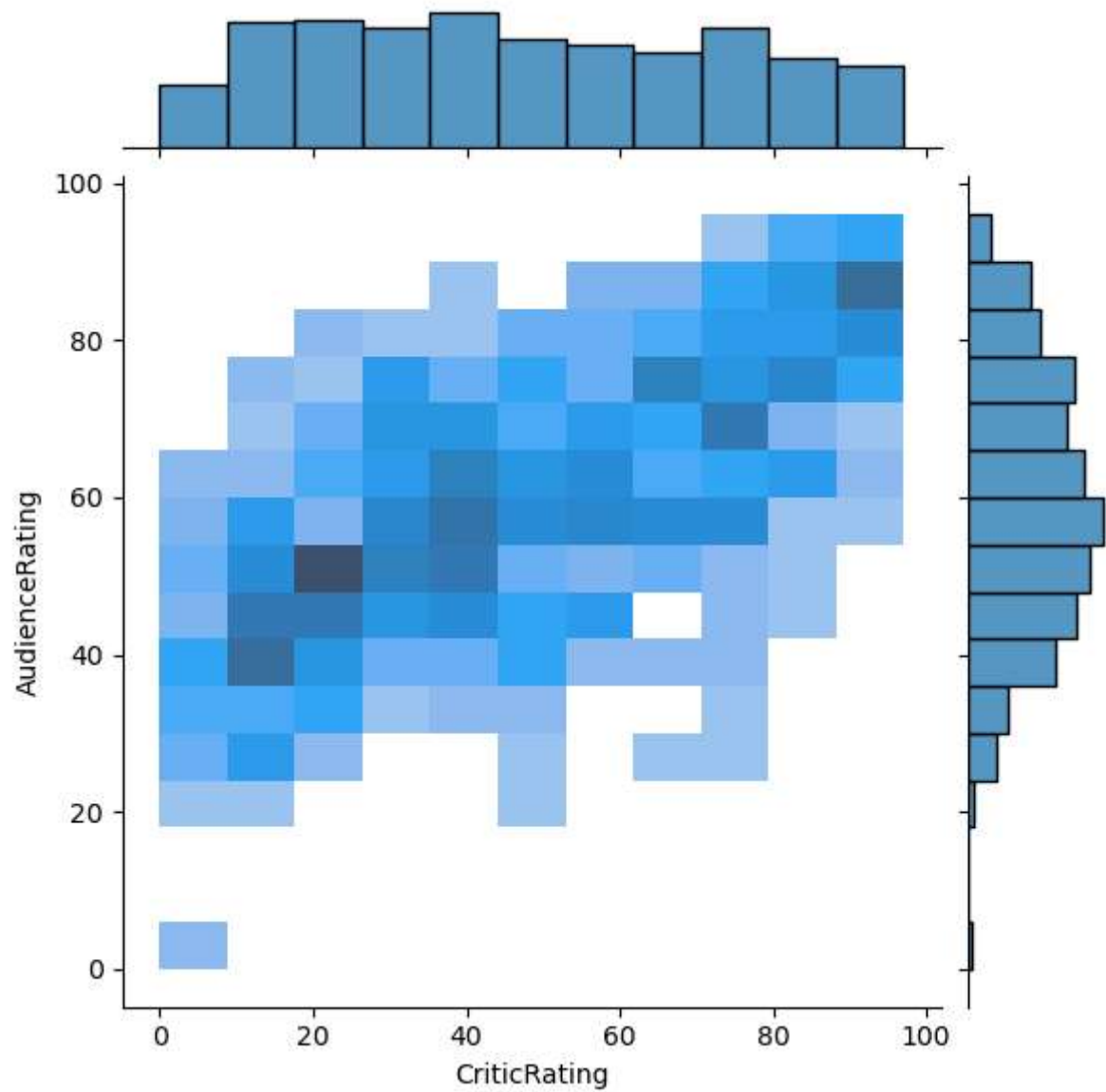
```
In [32]: j=sns.jointplot(data=movies, x='CriticRating', y='AudienceRating', kind='reg')
```



```
In [33]: j=sns.jointplot(data=movies, x='CriticRating', y='AudienceRating', kind='kde')
```

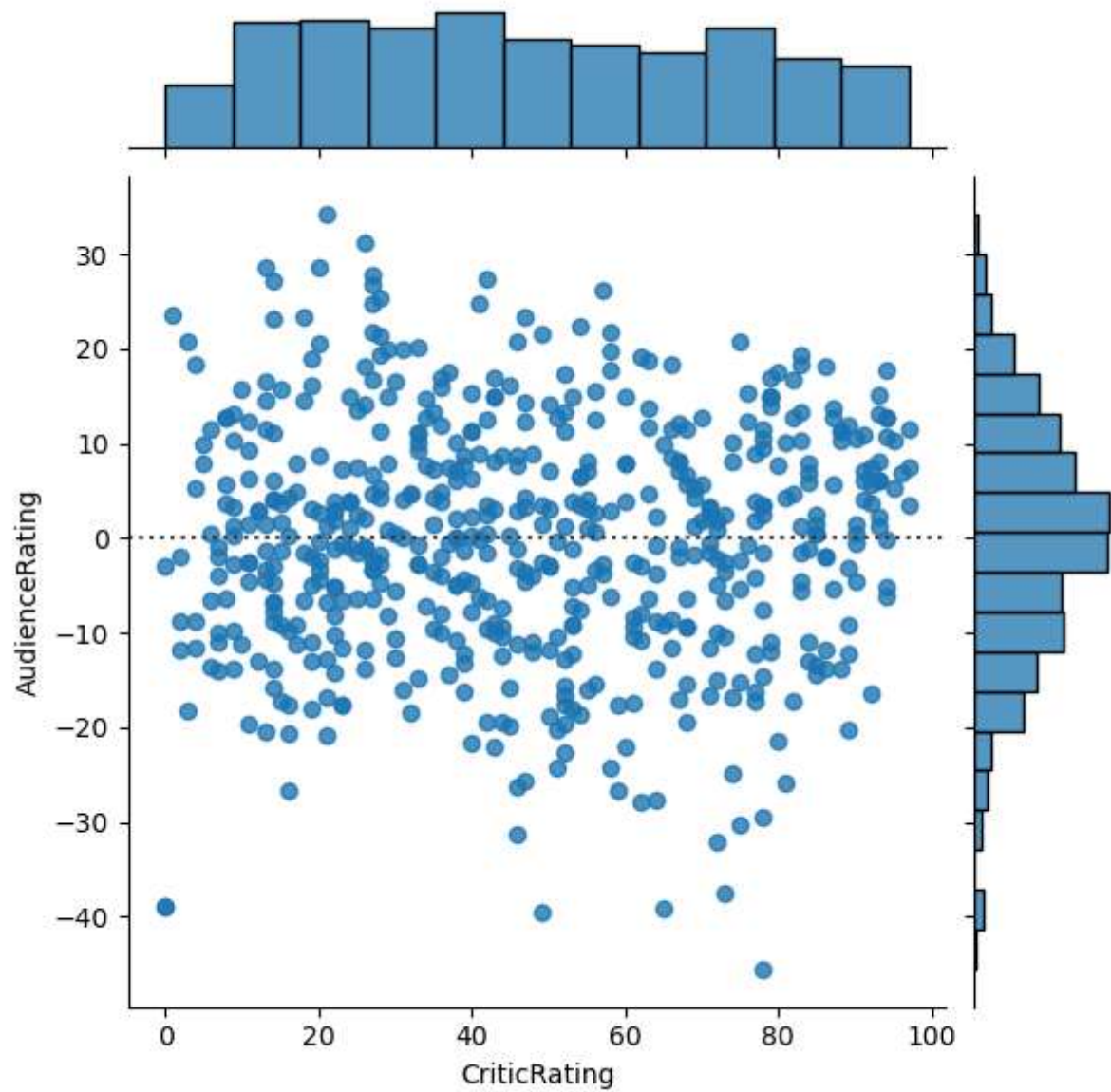


```
In [34]: j=sns.jointplot(data=movies, x='CriticRating', y='AudienceRating', kind='hist')
```



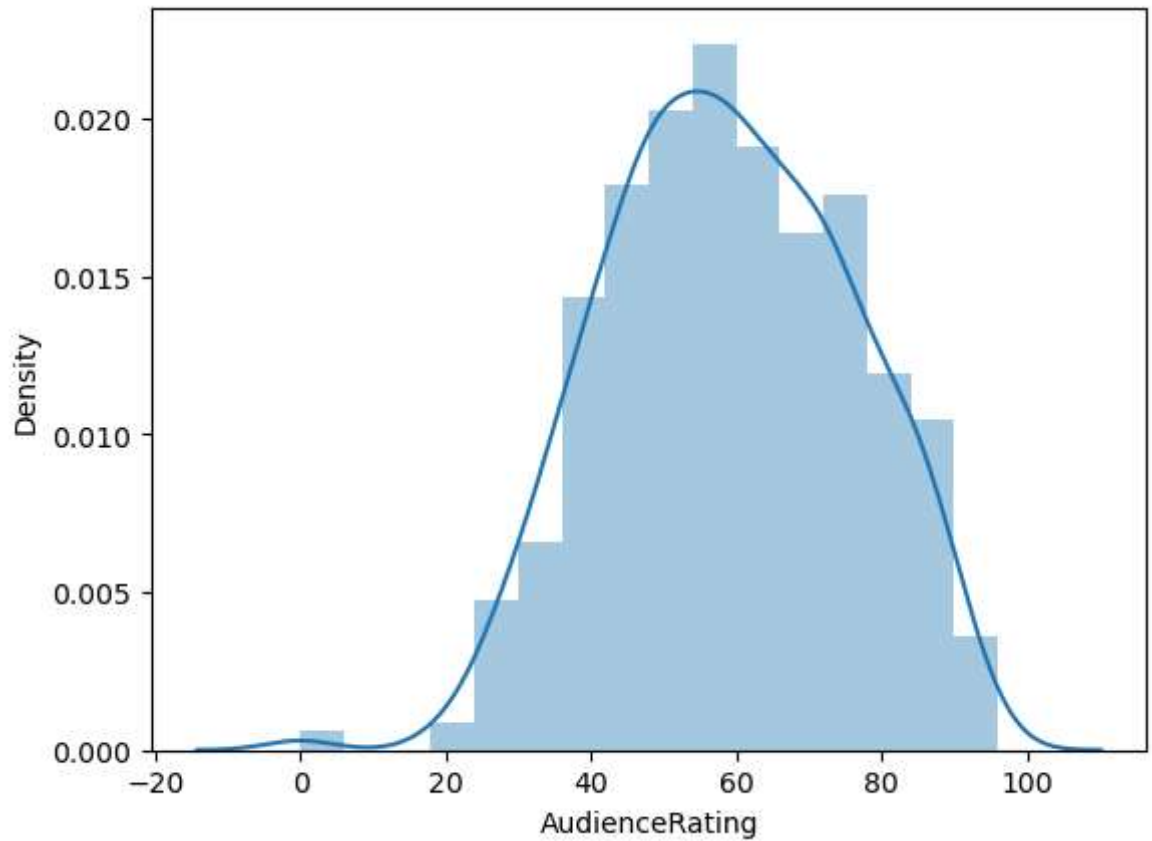
```
In [35]: j=sns.jointplot(data=movies, x='CriticRating', y='AudienceRating', kind='resid')
```





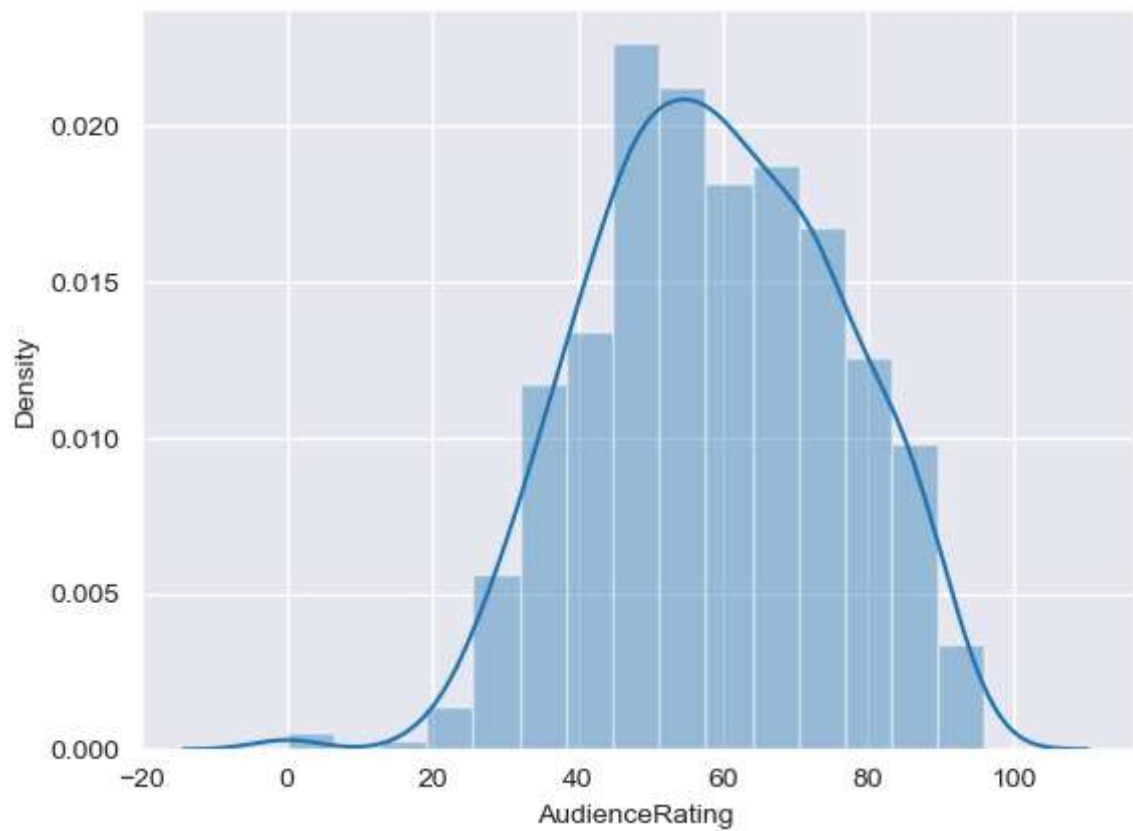
```
In [36]: # histograms  
# chat1
```

```
In [37]: m = sns.distplot(movies.AudienceRating)
```



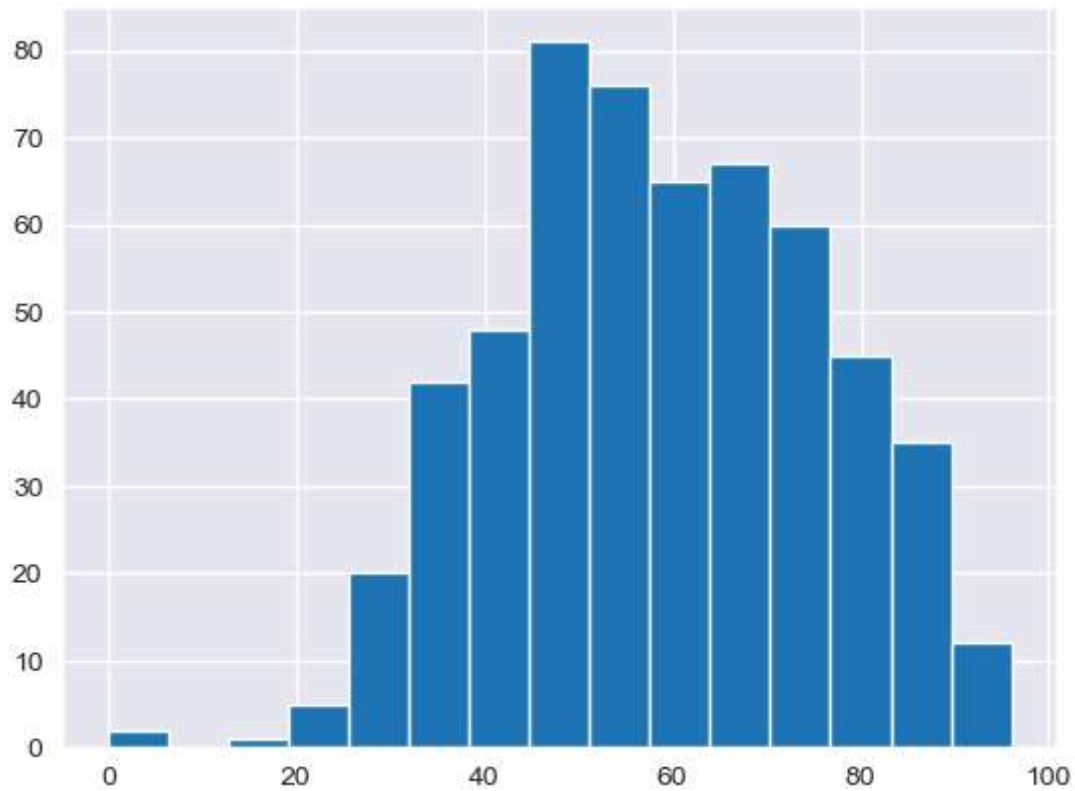
```
In [38]: sns.set_style('darkgrid')
```

```
In [39]: m1=sns.distplot(movies.AudienceRating,bins = 15)
```

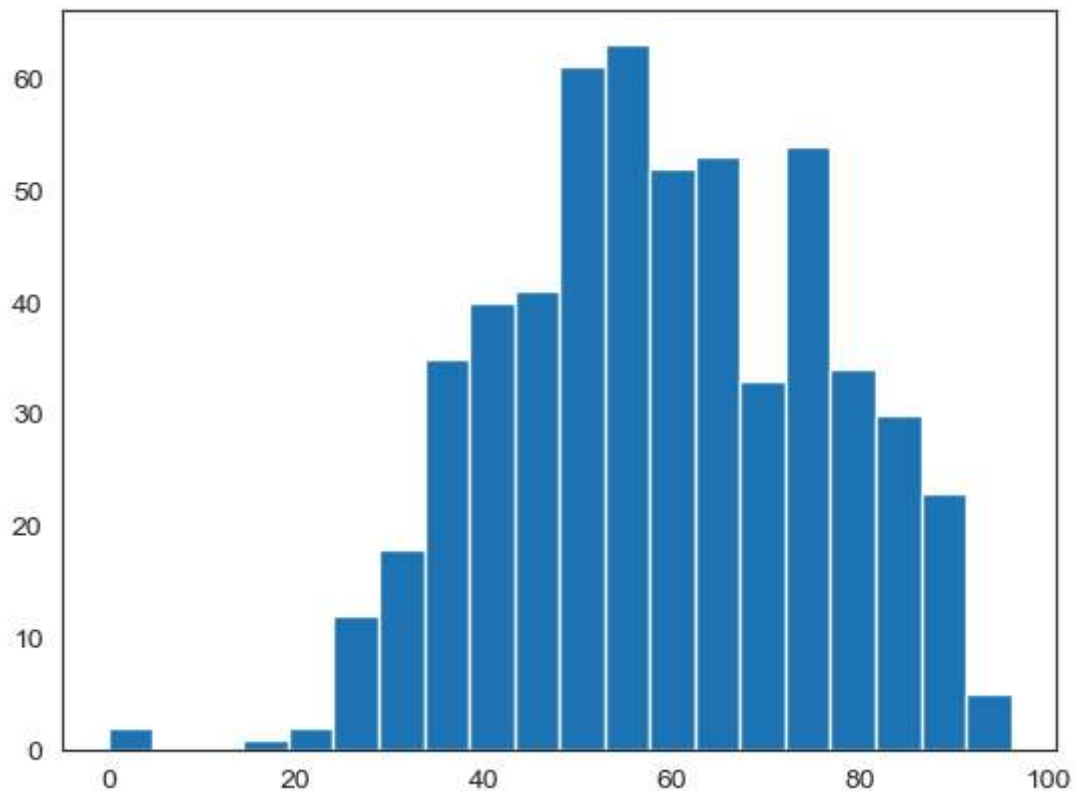




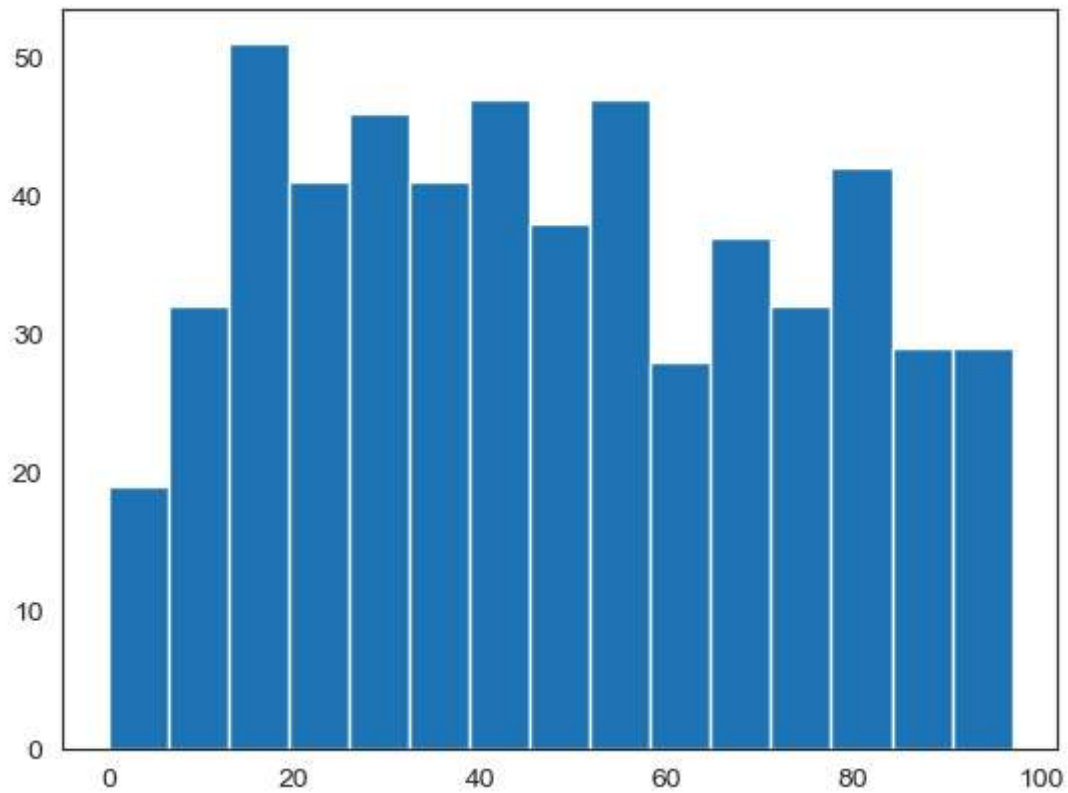
```
In [40]: n1 = plt.hist(movies.AudienceRating, bins=15)
```



```
In [41]: sns.set_style('white')  
n1 = plt.hist(movies.AudienceRating, bins=20)
```

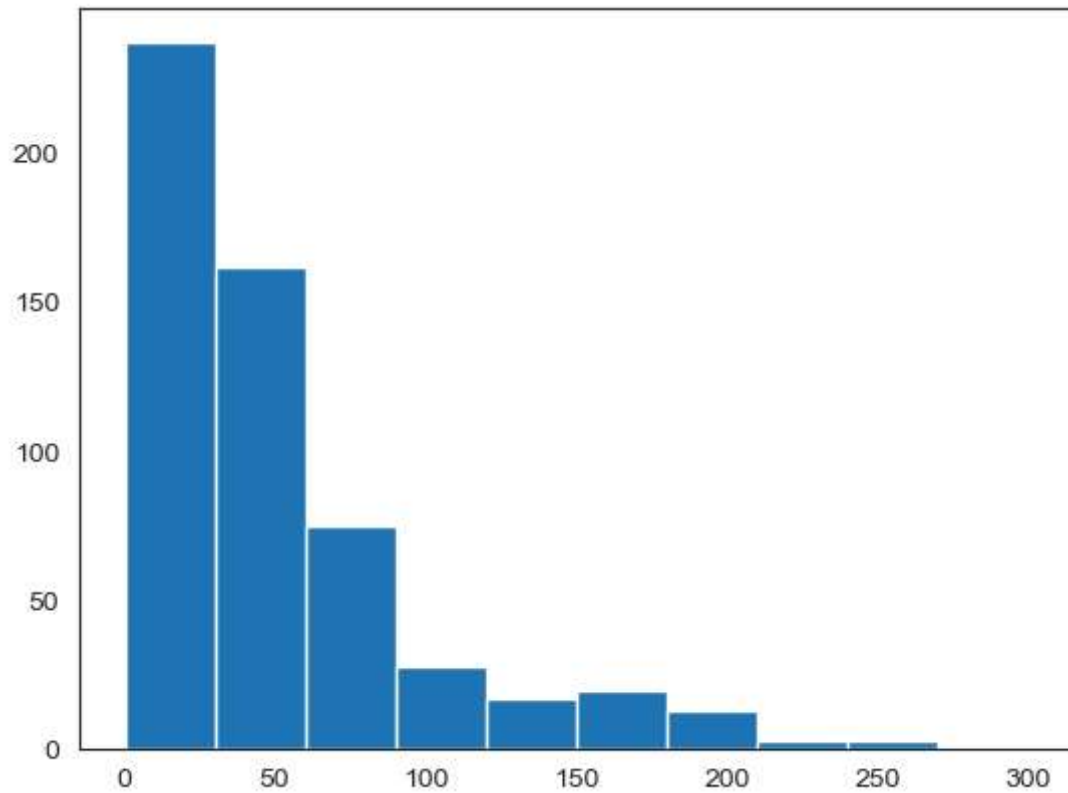


```
In [42]: n1 = plt.hist(movies.CriticRating, bins=15)
```

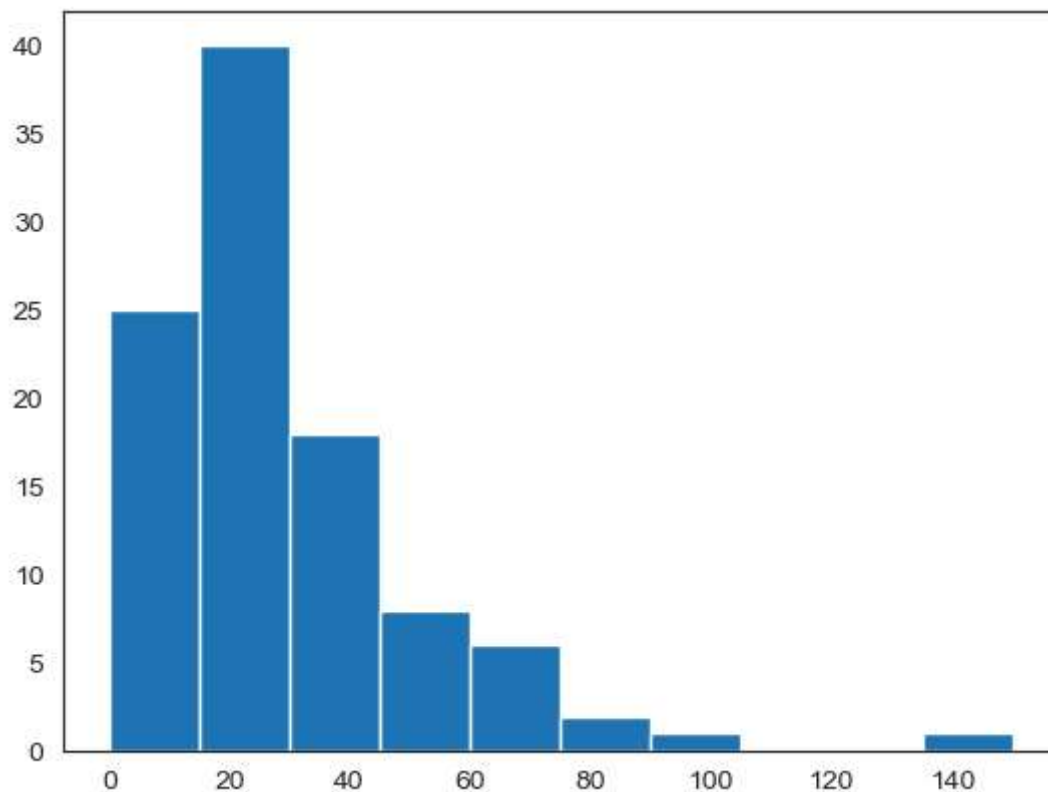


```
In [43]: # chat2
```

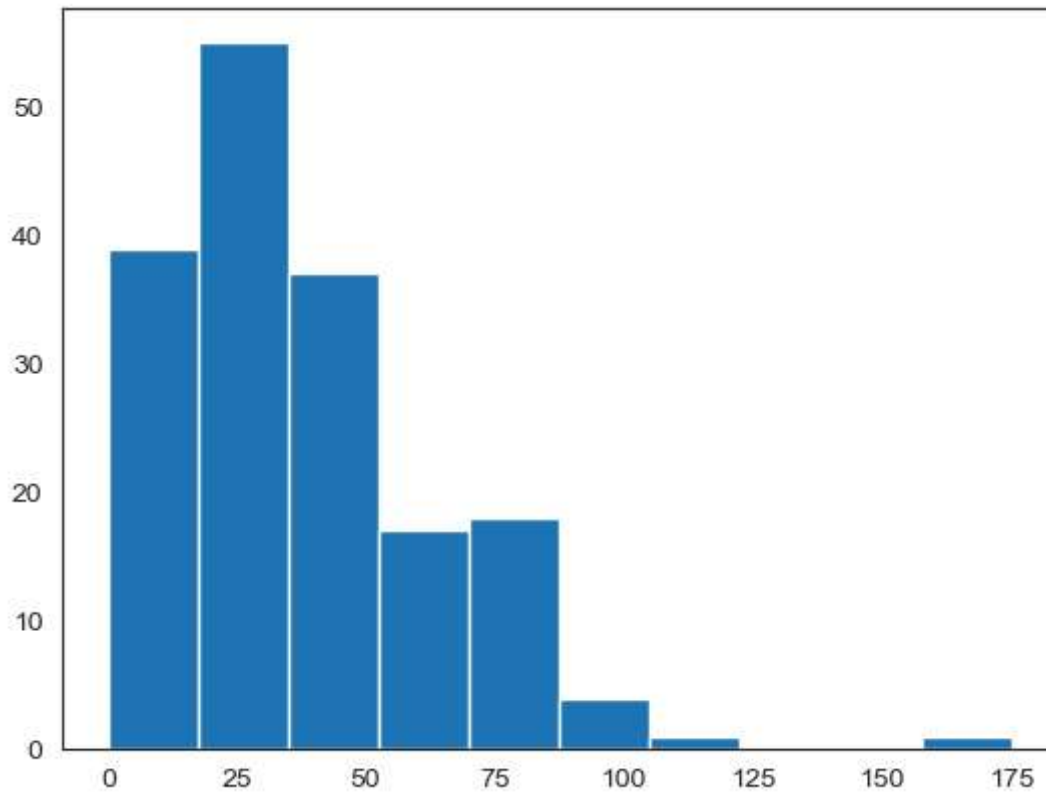
```
In [44]: plt.hist(movies.BudgetMillions)  
plt.show()
```



```
In [45]: plt.hist(movies[movies.Genre == 'Drama'].BudgetMillions)
plt.show()
```

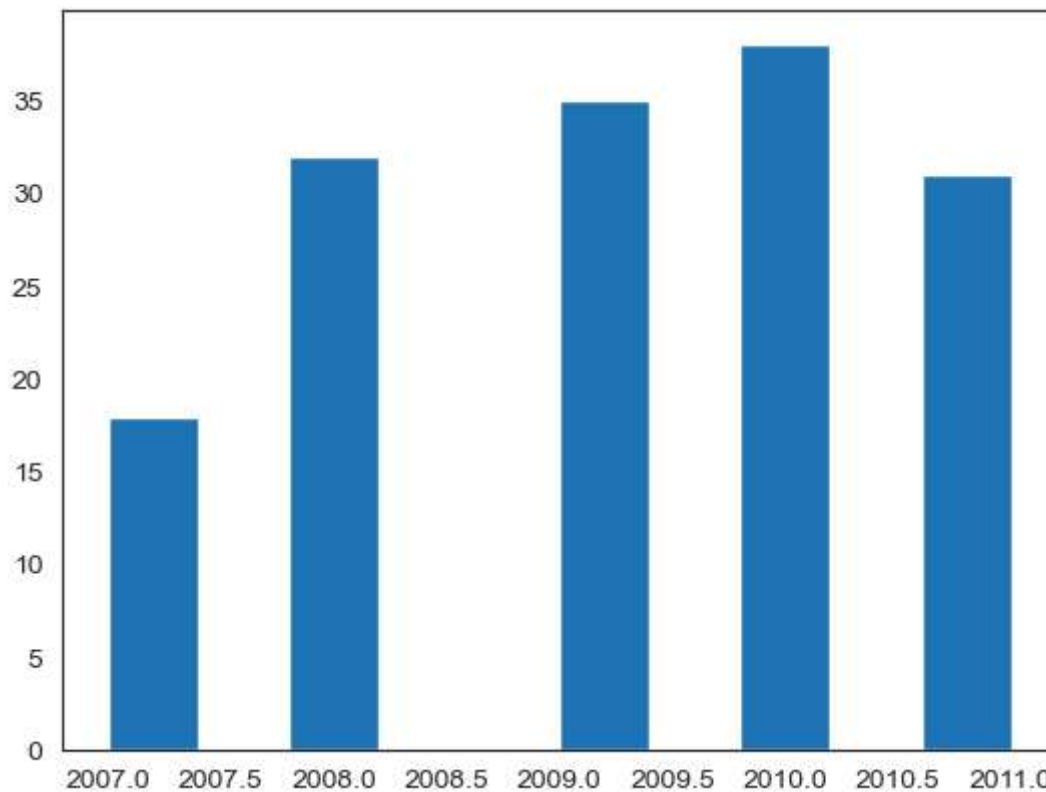


```
In [46]: plt.hist(movies[movies.Genre == 'Comedy'].BudgetMillions)
plt.show()
```



```
In [47]: plt.hist(movies[movies.Genre == 'Action'].Year)
```

```
Out[47]: (array([18.,  0., 32.,  0.,  0., 35.,  0., 38.,  0., 31.]),  
          array([2007. , 2007.4, 2007.8, 2008.2, 2008.6, 2009. , 2009.4, 2009.8,  
                2010.2, 2010.6, 2011. ]),  
          <BarContainer object of 10 artists>)
```



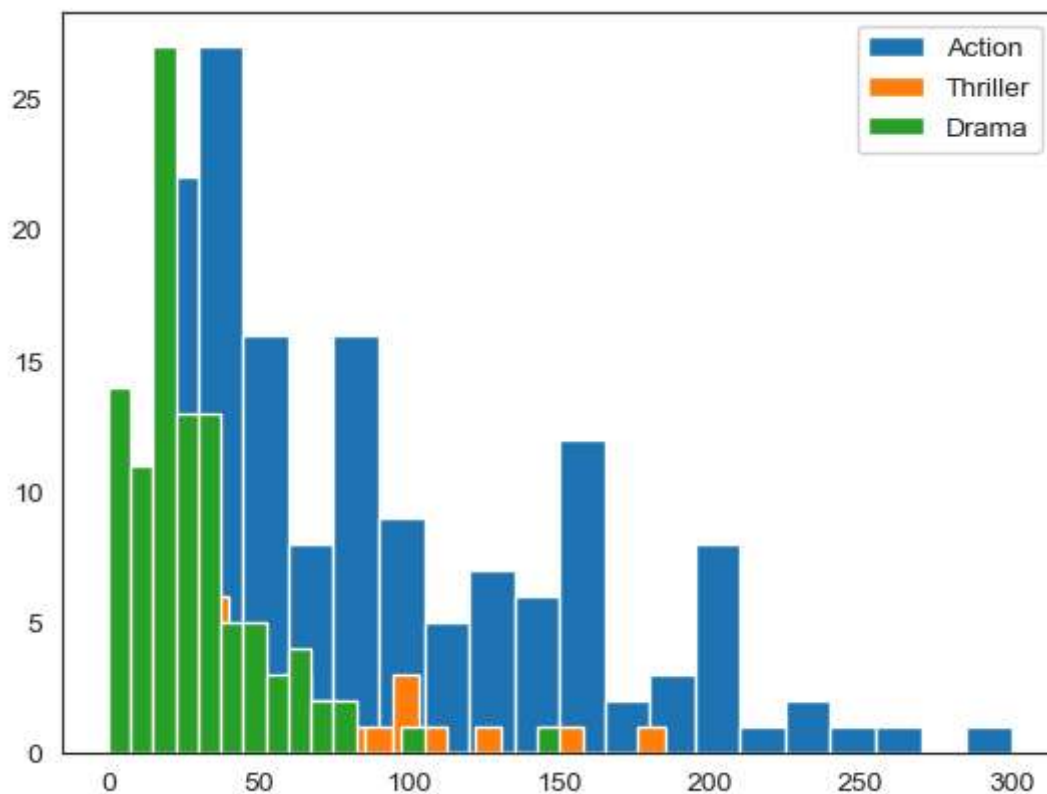
```
In [48]: movies.head()
```

```
Out[48]:
```

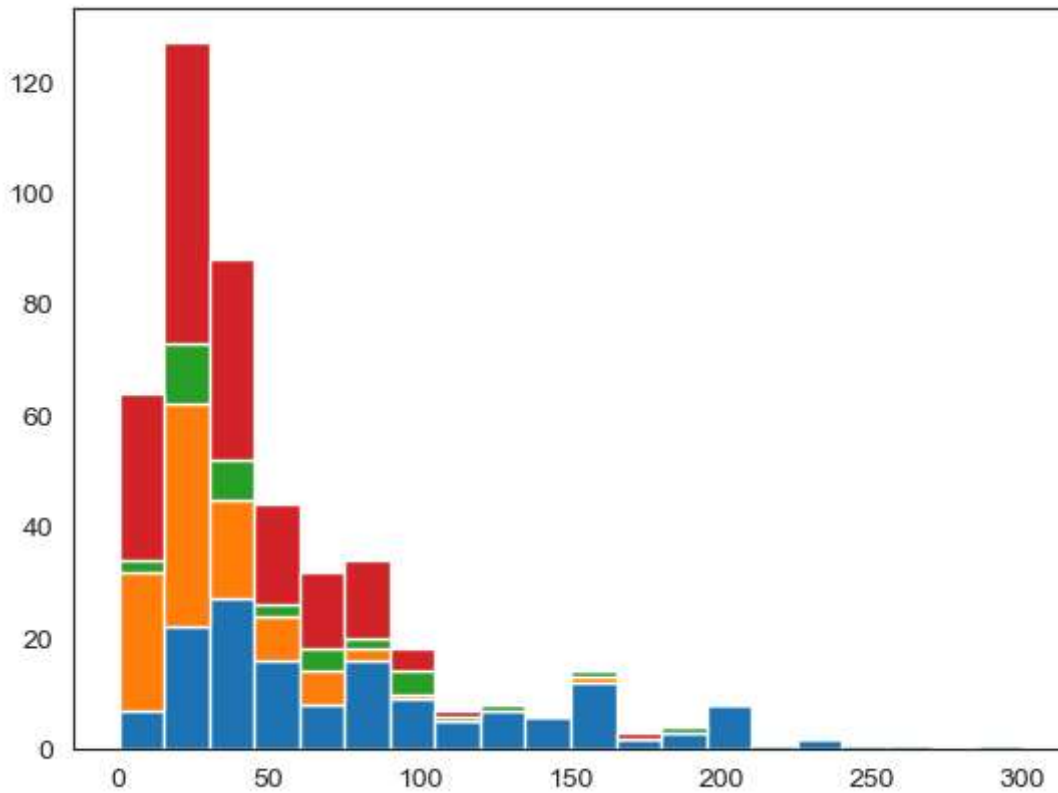
|   | Film                 | Genre     | CriticRating | AudienceRating | BudgetMillions | Year |
|---|----------------------|-----------|--------------|----------------|----------------|------|
| 0 | (500) Days of Summer | Comedy    | 87           | 81             | 8              | 2009 |
| 1 | 10,000 B.C.          | Adventure | 9            | 44             | 105            | 2008 |
| 2 | 12 Rounds            | Action    | 30           | 52             | 20             | 2009 |
| 3 | 127 Hours            | Adventure | 93           | 84             | 18             | 2010 |
| 4 | 17 Again             | Comedy    | 55           | 70             | 20             | 2009 |

```
In [49]: # Below plots are stacked histogram becuae overlaped

plt.hist(movies[movies.Genre == 'Action'].BudgetMillions, bins = 20, label='Action')
plt.hist(movies[movies.Genre == 'Thriller'].BudgetMillions, bins = 20, label='Thrill')
plt.hist(movies[movies.Genre == 'Drama'].BudgetMillions, bins = 20, label='Drama')
plt.legend()
plt.show()
```



```
In [50]: plt.hist([movies[movies.Genre == 'Action'].BudgetMillions, \
                    movies[movies.Genre == 'Drama'].BudgetMillions, \
                    movies[movies.Genre == 'Thriller'].BudgetMillions, \
                    movies[movies.Genre == 'Comedy'].BudgetMillions],
                bins = 20, stacked = True)
plt.show()
```

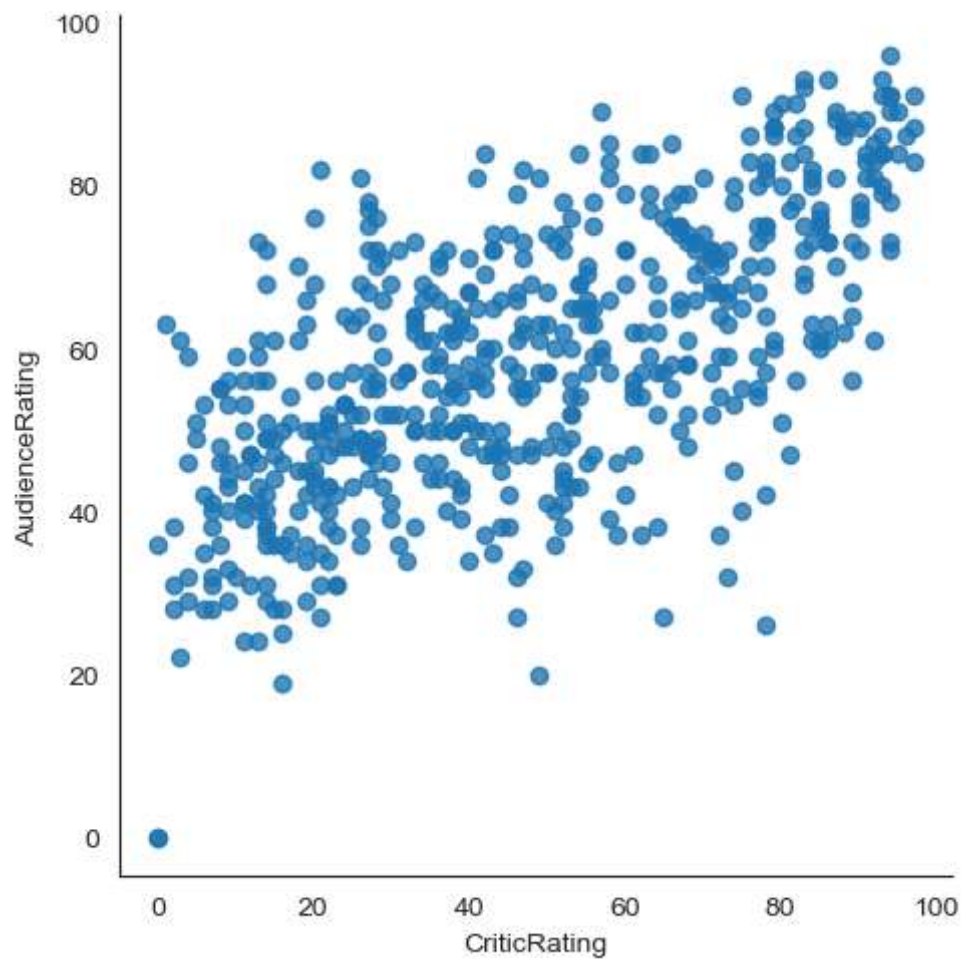


In [51]: *# if you have 100 categories you cannot copy & paste all the things*

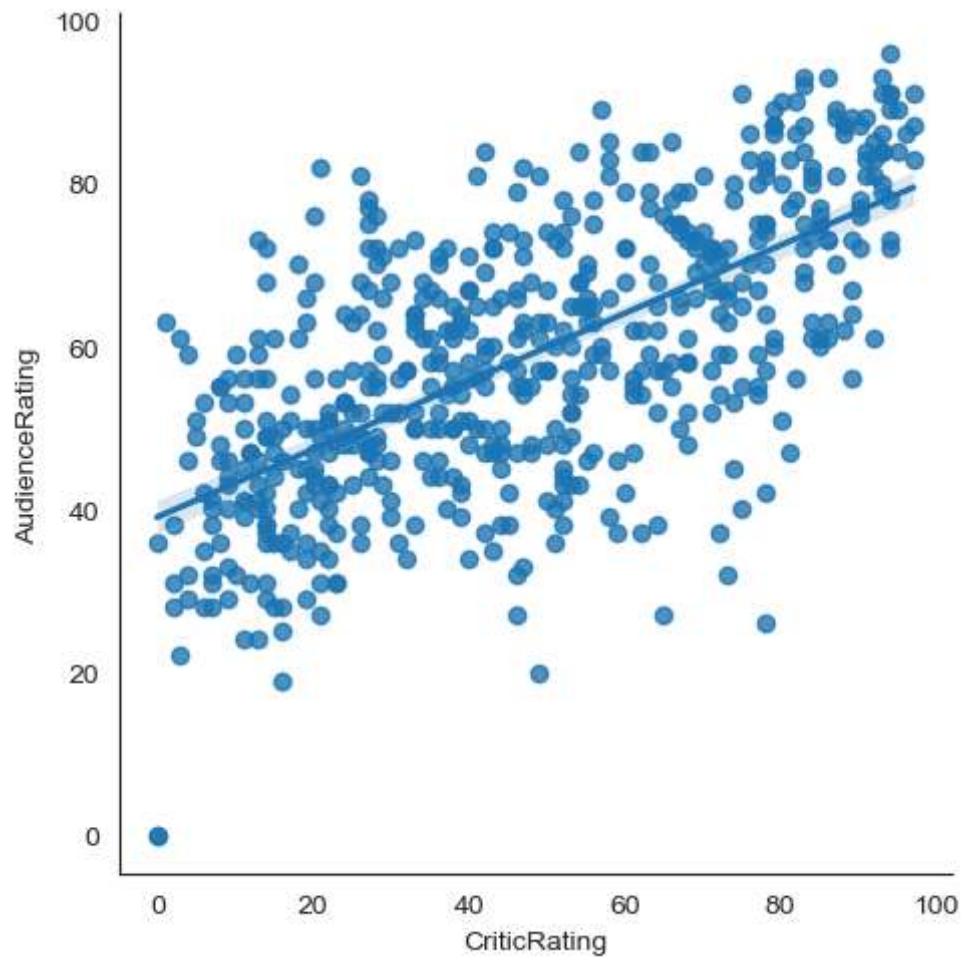
```
for gen in movies.Genre.cat.categories:  
    print(gen)
```

Action  
Adventure  
Comedy  
Drama  
Horror  
Romance  
Thriller

In [52]: `vis1 = sns.lmplot(data=movies, x='CriticRating', y='AudienceRating',\n fit_reg=False)`

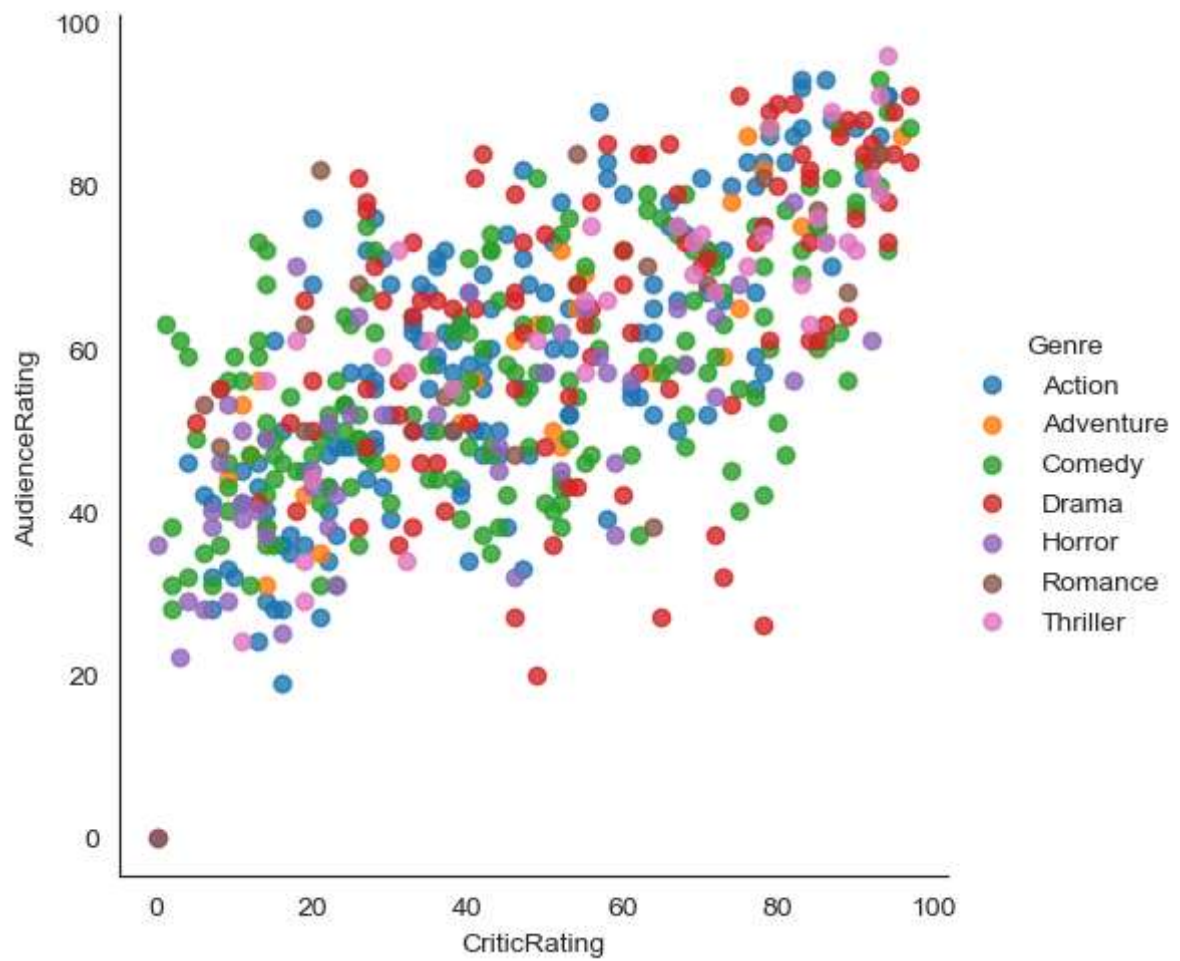


```
In [53]: vis1 = sns.lmplot(data=movies, x='CriticRating', y='AudienceRating',\n                           fit_reg=True)
```

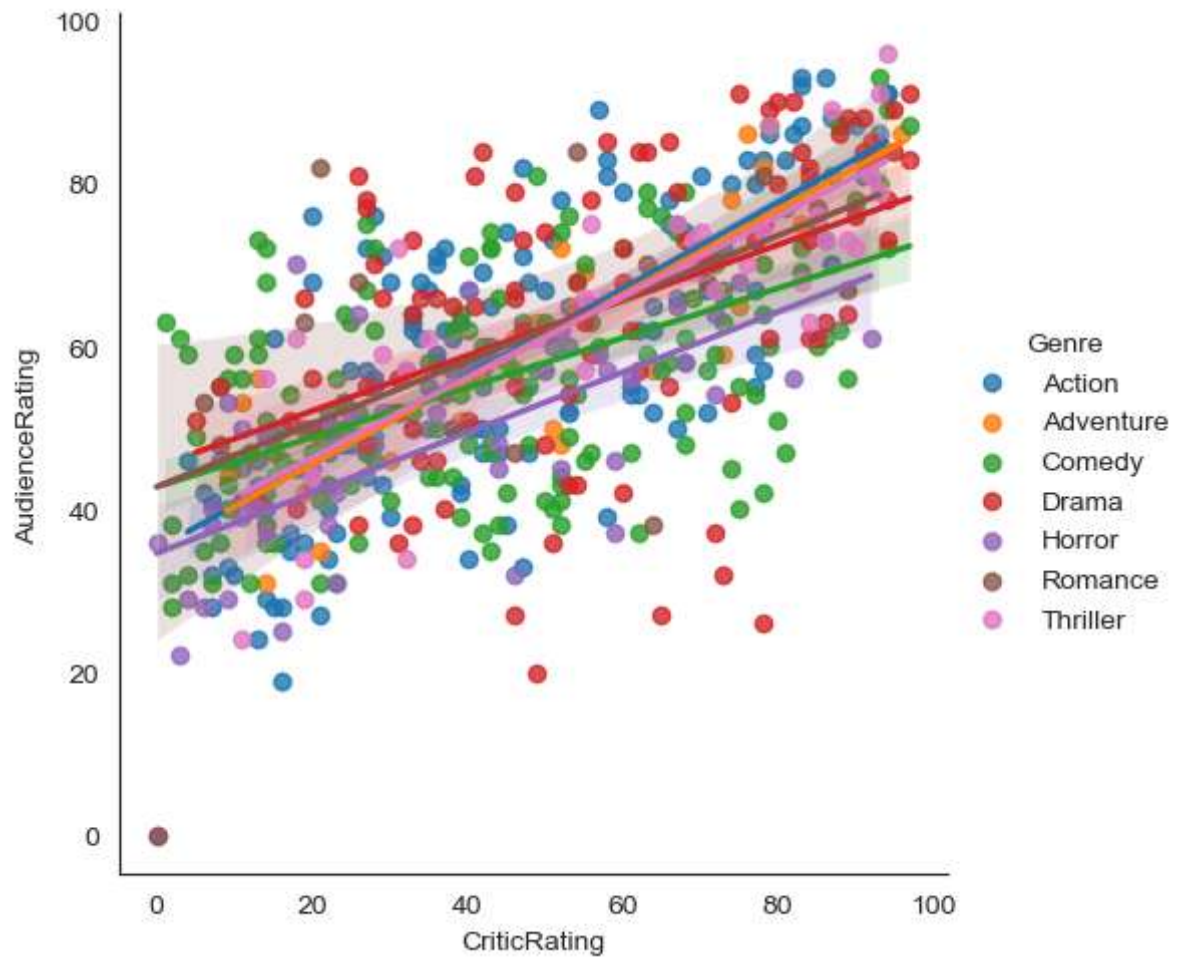


```
In [54]: vis1 = sns.lmplot(data=movies, x='CriticRating', y='AudienceRating',\n                           fit_reg=False, hue = 'Genre')
```



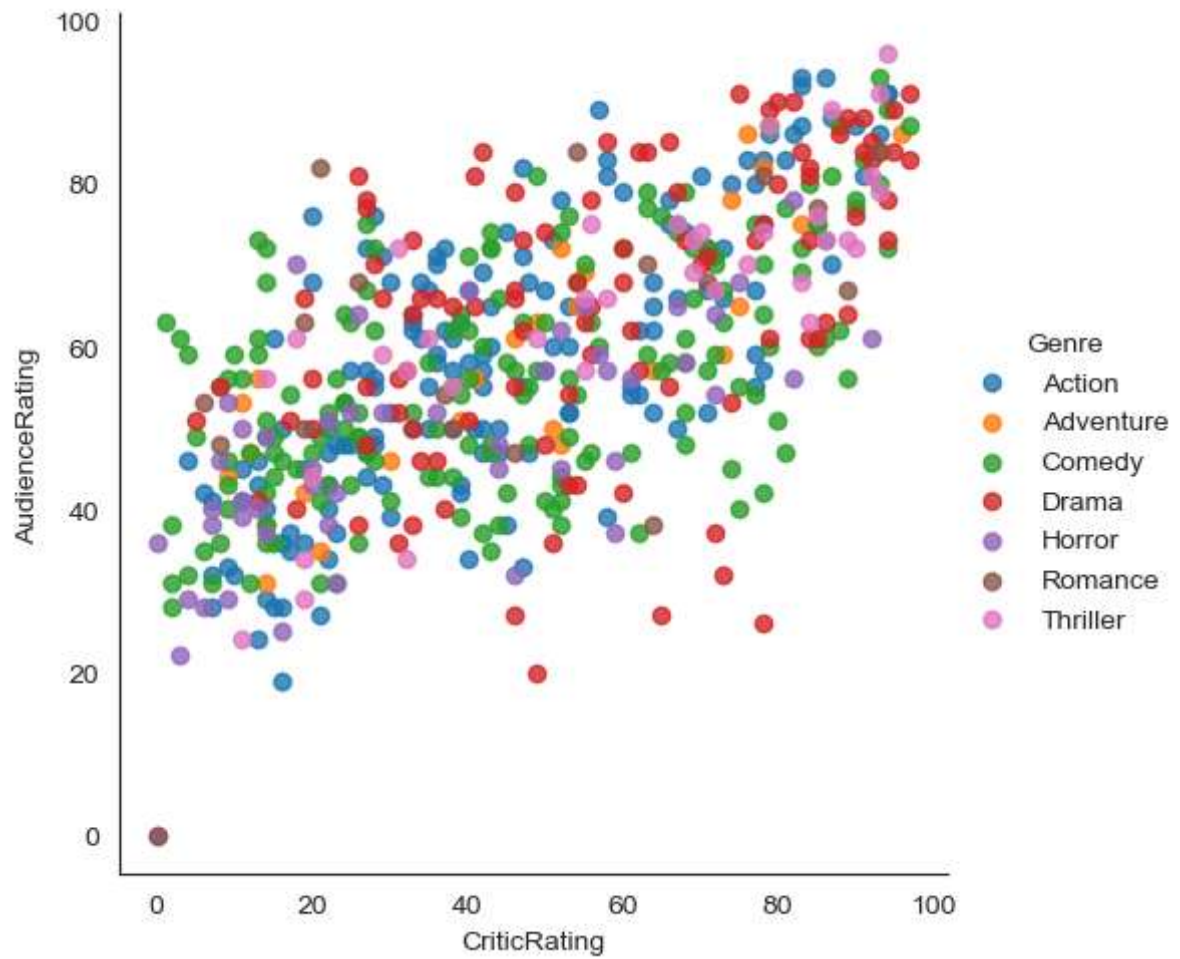


```
In [55]: vis1 = sns.lmplot(data=movies, x='CriticRating', y='AudienceRating',\n                           fit_reg=True, hue = 'Genre')
```



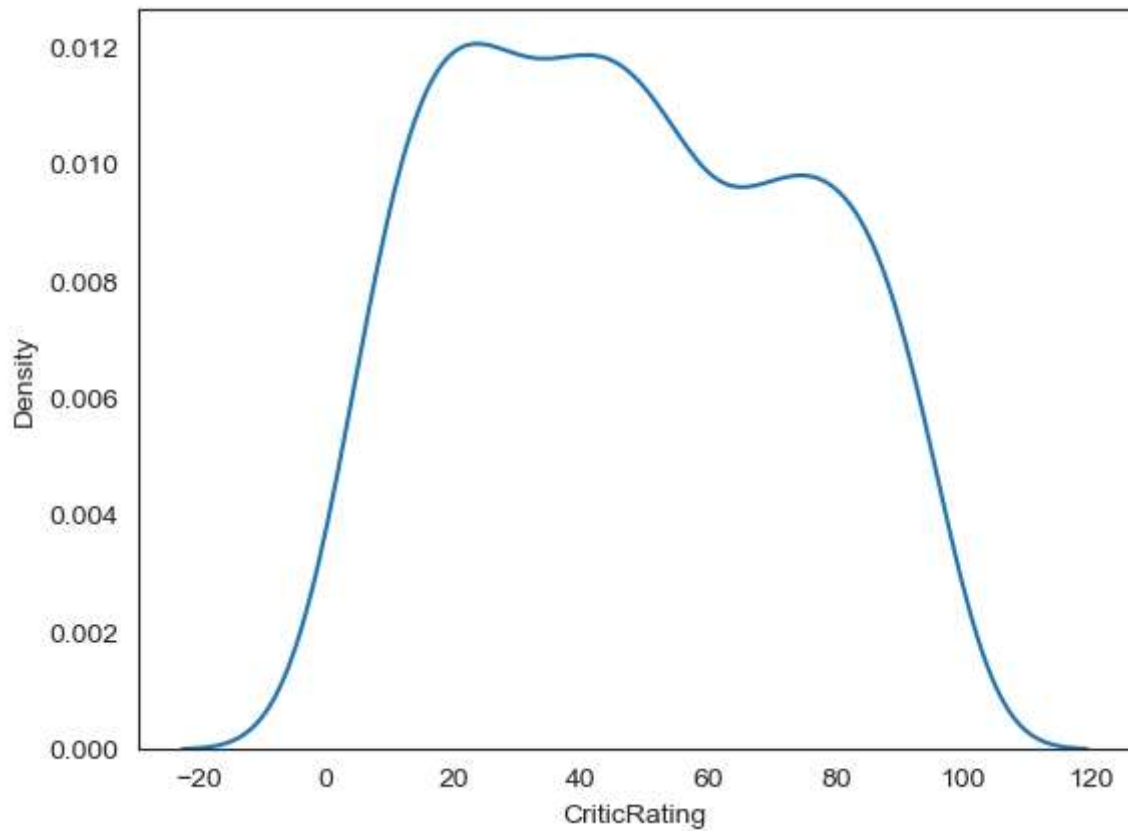
```
vis1 = sns.lmplot(data=movies, x='CriticRating', y='AudienceRating',\ fit_reg=False, hue = 'Genre',size=10,aspect=1)
```

```
In [56]: vis1 = sns.lmplot(data=movies, x='CriticRating', y='AudienceRating',\  
                           fit_reg=False, hue = 'Genre',aspect=1)
```

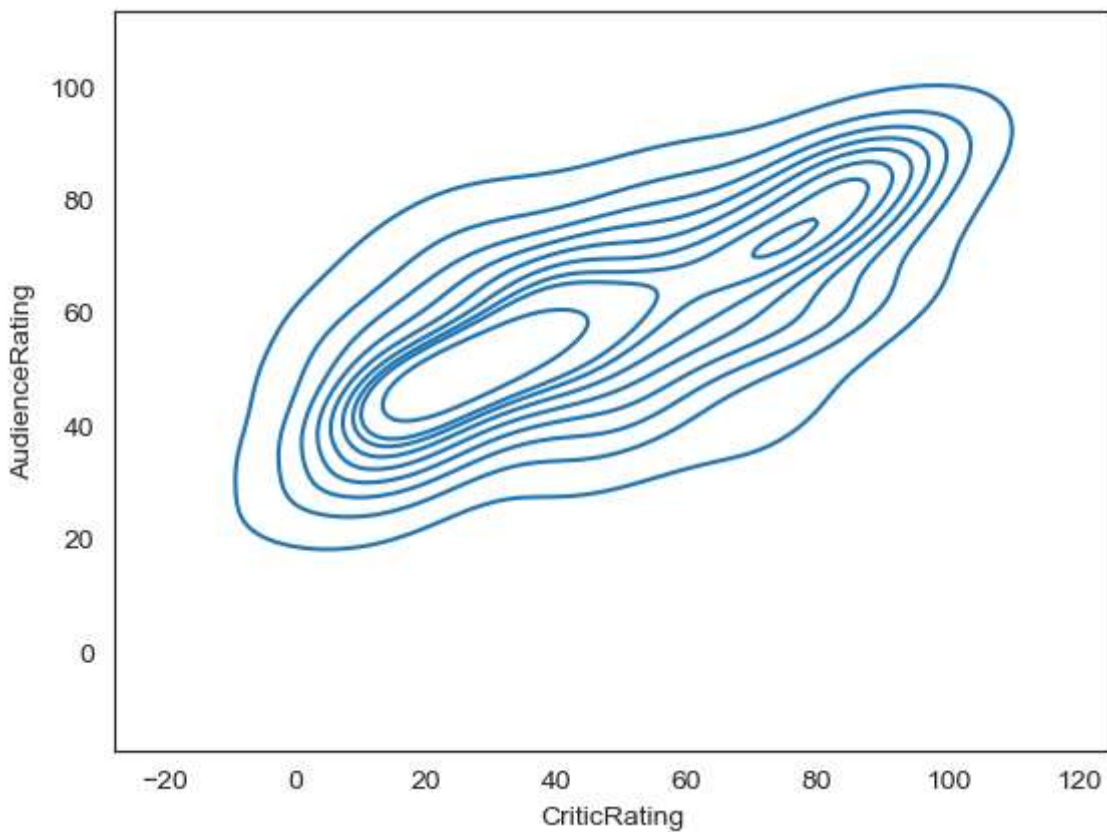


# Kernel Density Estimate plot ( KDE PLOT) # how can i visualize audience rating & critics rating . using scatterplot

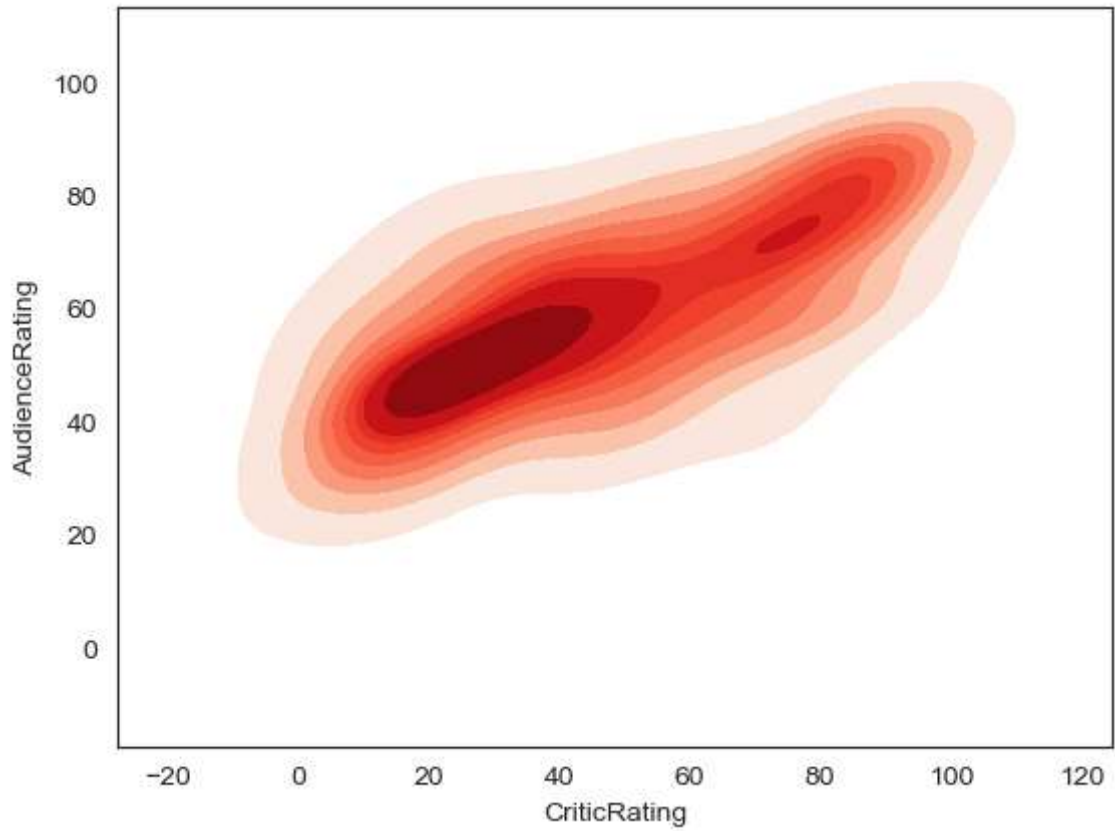
```
In [57]: k1 = sns.kdeplot(movies.CriticRating)
```



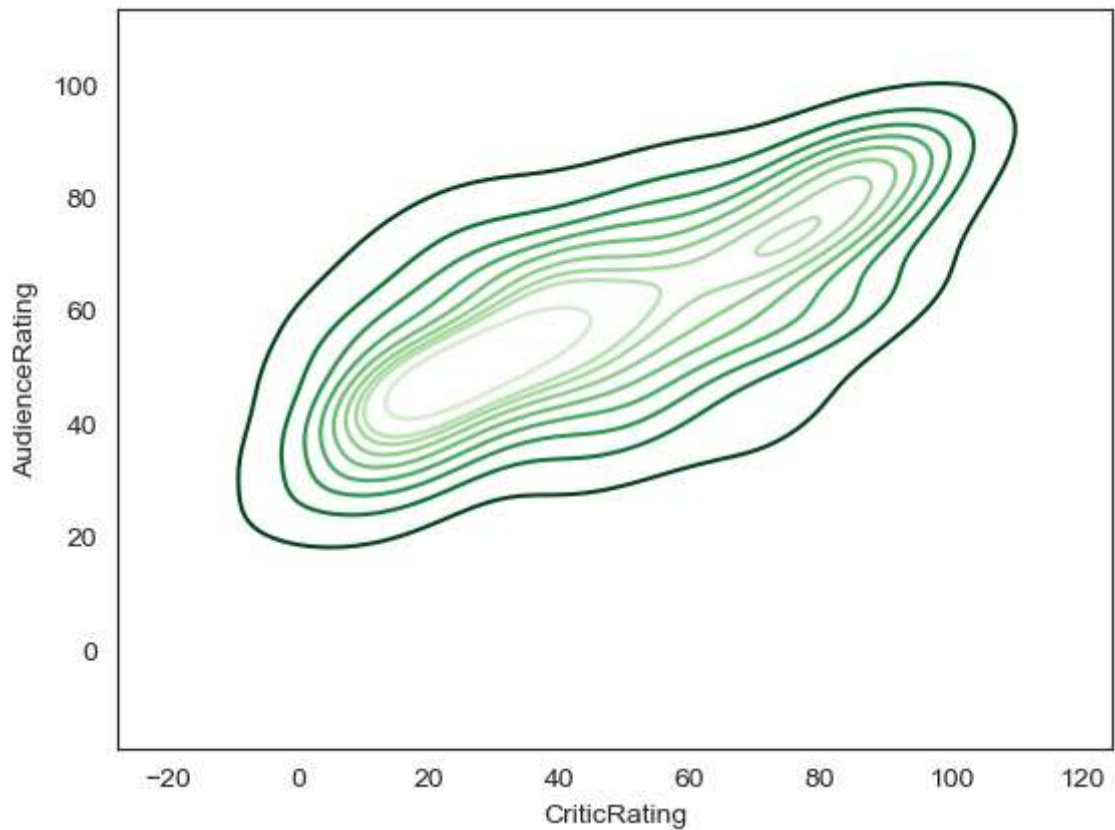
```
In [58]: k1 = sns.kdeplot(x=movies.CriticRating,y=movies.AudienceRating)
```



```
In [59]: k1 = sns.kdeplot(movies,x='CriticRating',y='AudienceRating',shade = True,shade_lowe
```

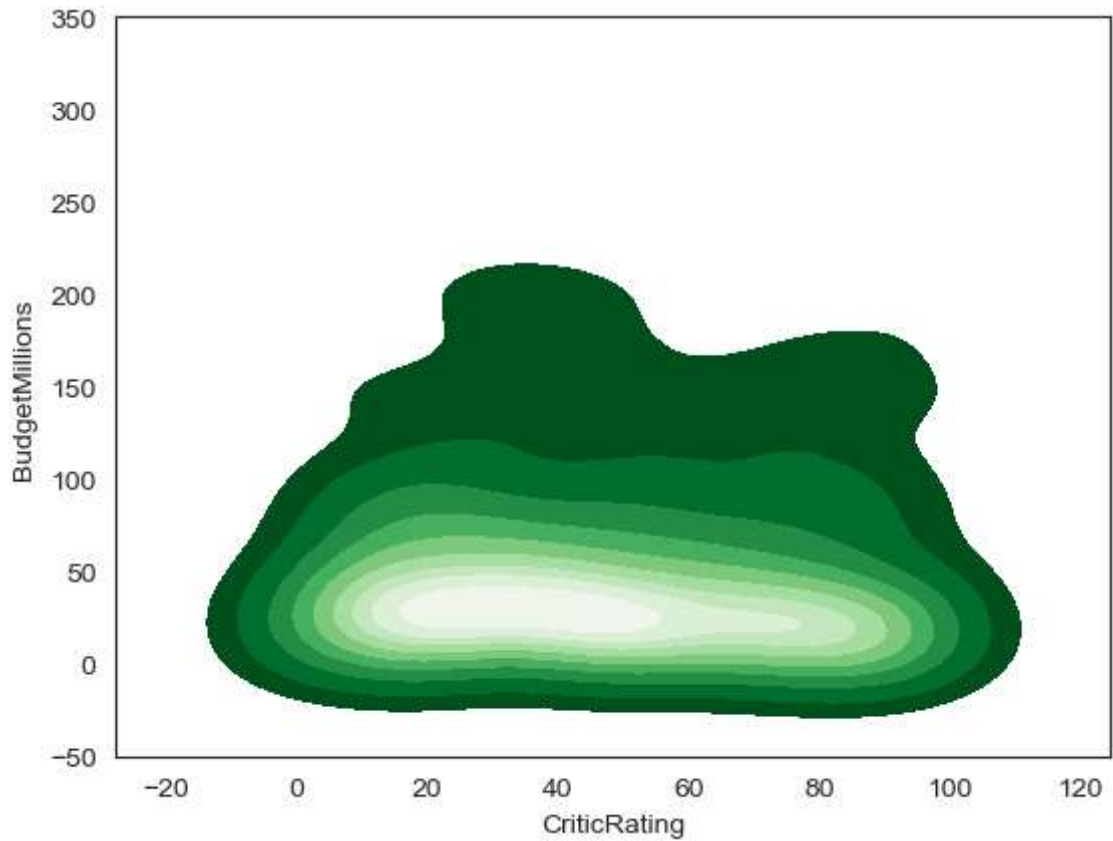


```
In [60]: k2 = sns.kdeplot(movies,x='CriticRating',y='AudienceRating',shade_lowest=False,cmap
```

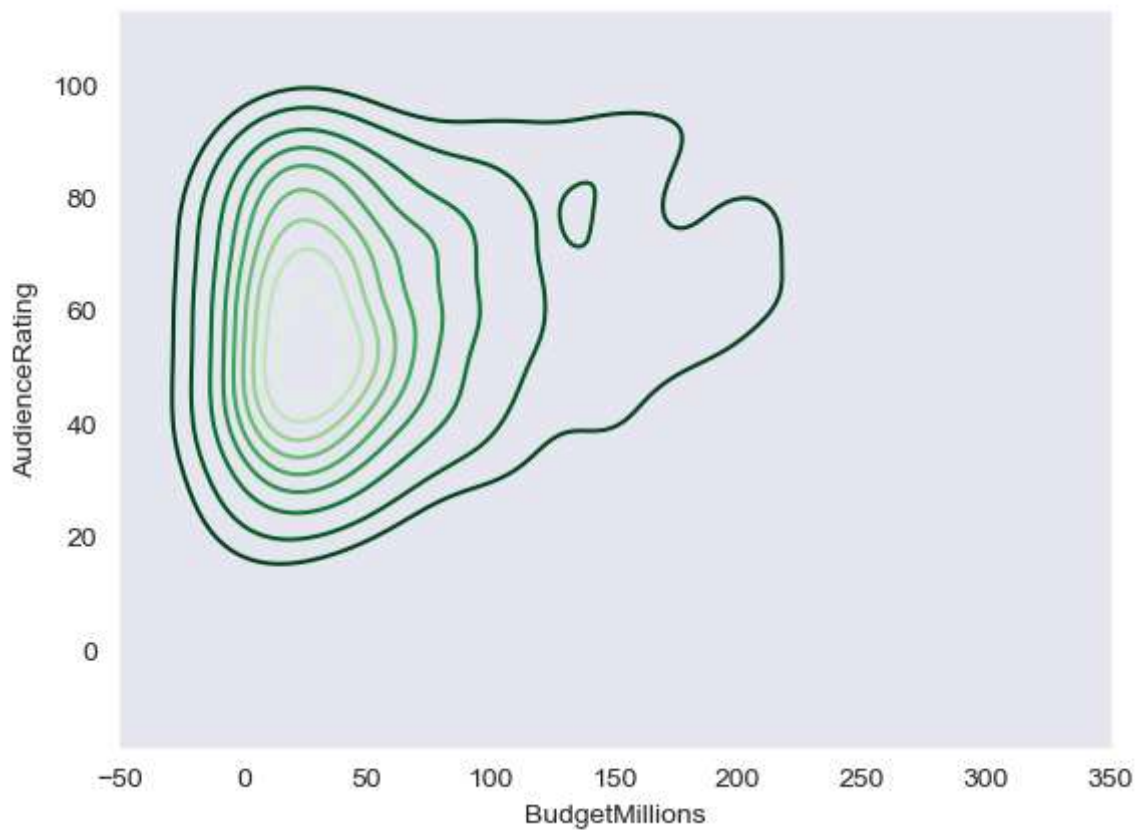


```
In [61]: k2 = sns.kdeplot(x=movies.CriticRating,y=movies.BudgetMillions,shade=True,cmap='Gre
```

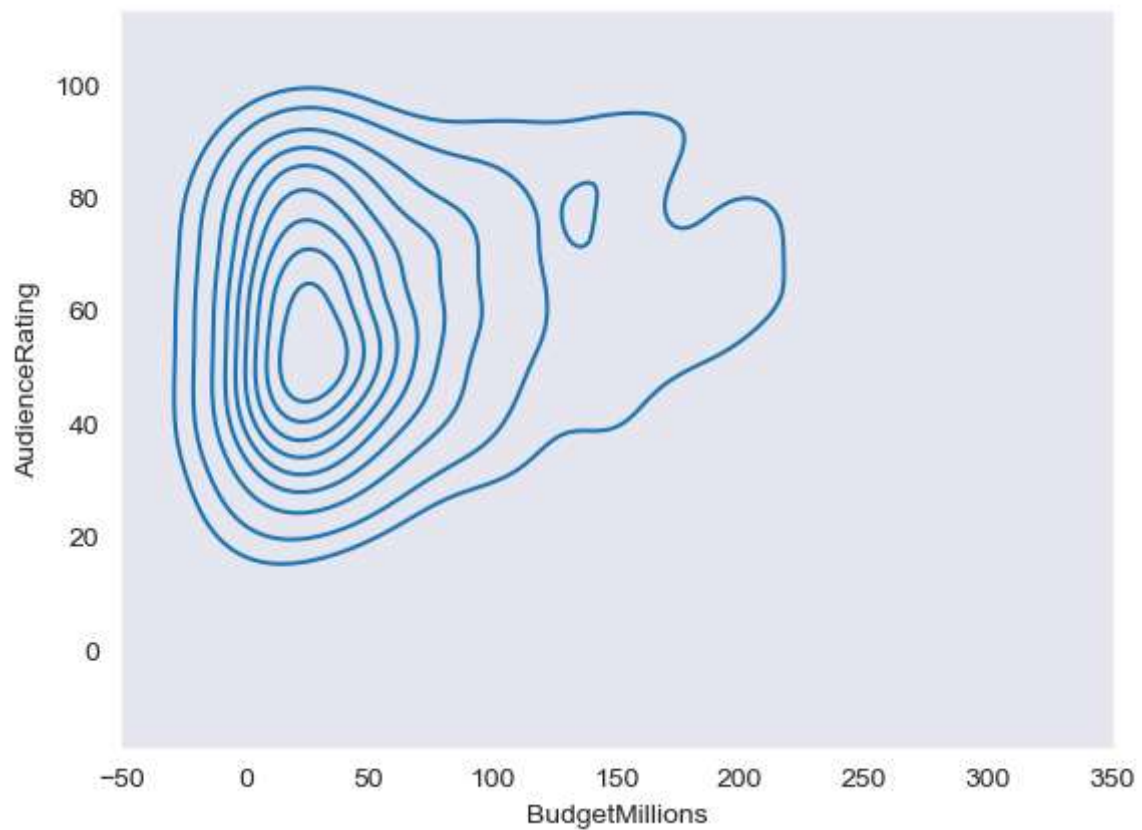




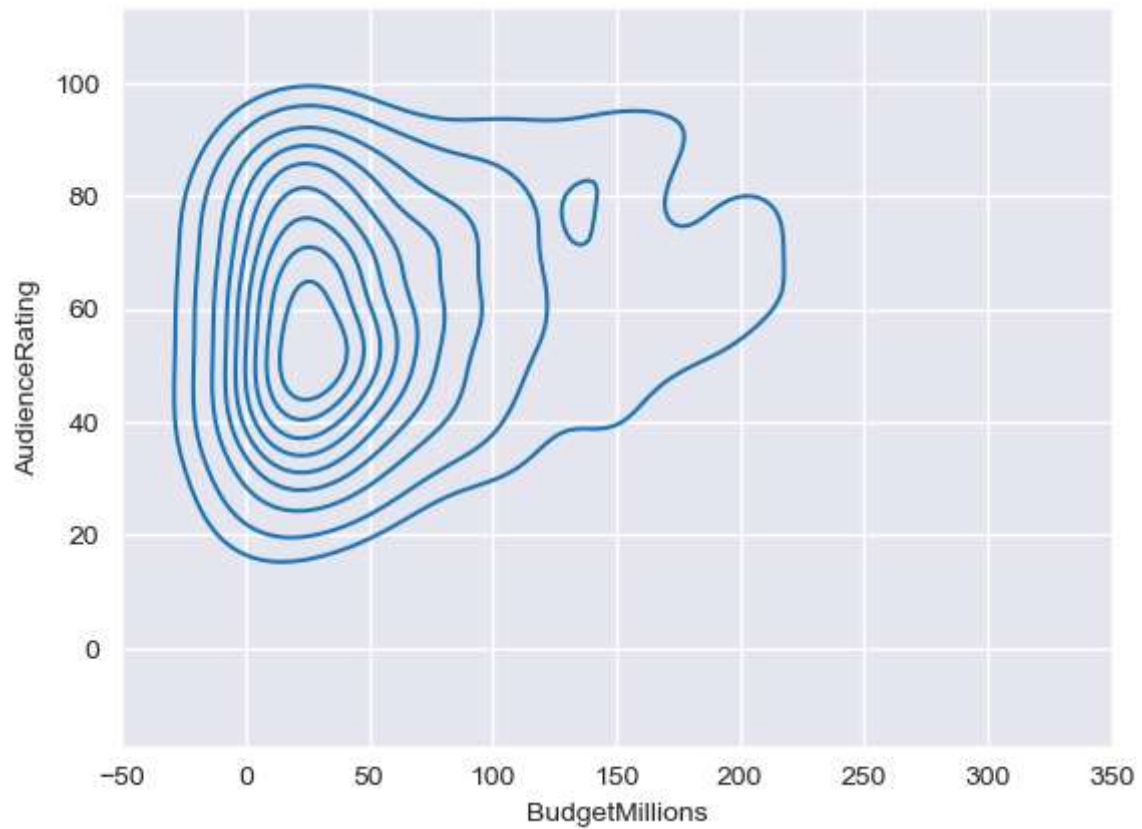
```
In [62]: sns.set_style('dark')  
k1 = sns.kdeplot(movies,x='BudgetMillions',y='AudienceRating',shade_lowest=False,cm
```



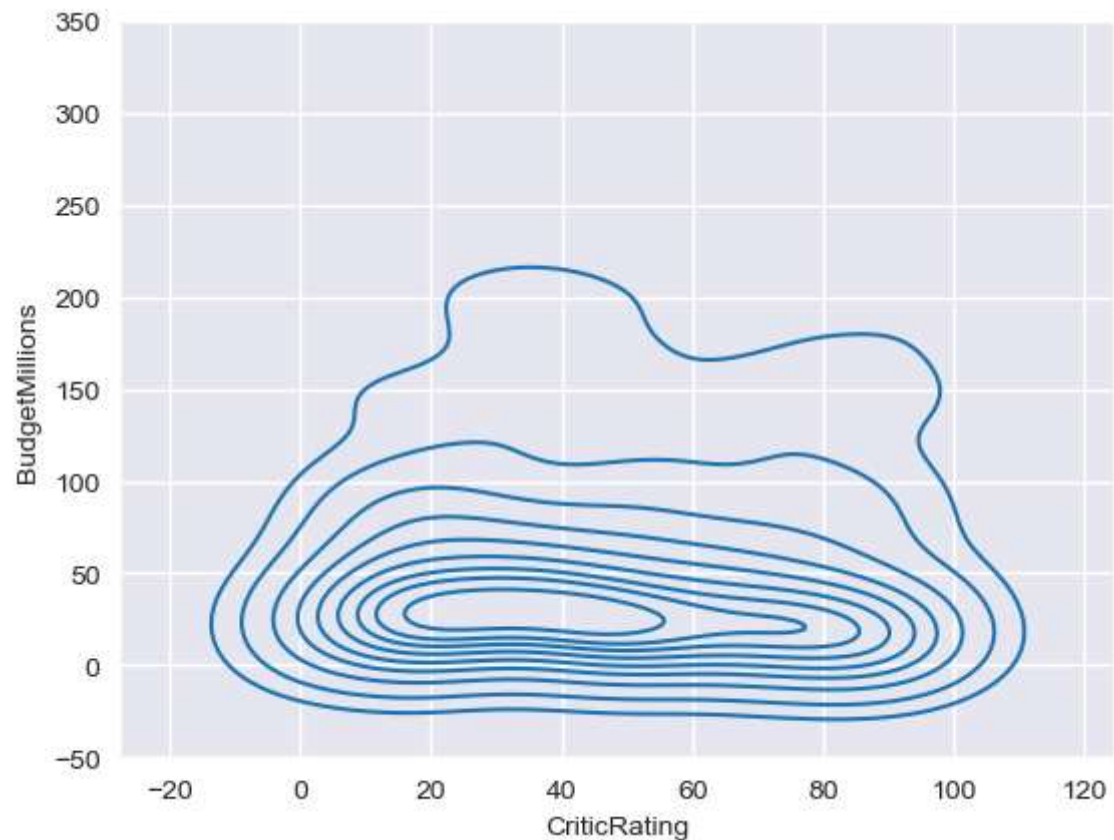
```
In [63]: sns.set_style('dark')  
k1 = sns.kdeplot(movies,x='BudgetMillions',y='AudienceRating')
```



```
In [64]: sns.set_style('darkgrid')  
k1 = sns.kdeplot(movies,x='BudgetMillions',y='AudienceRating')
```

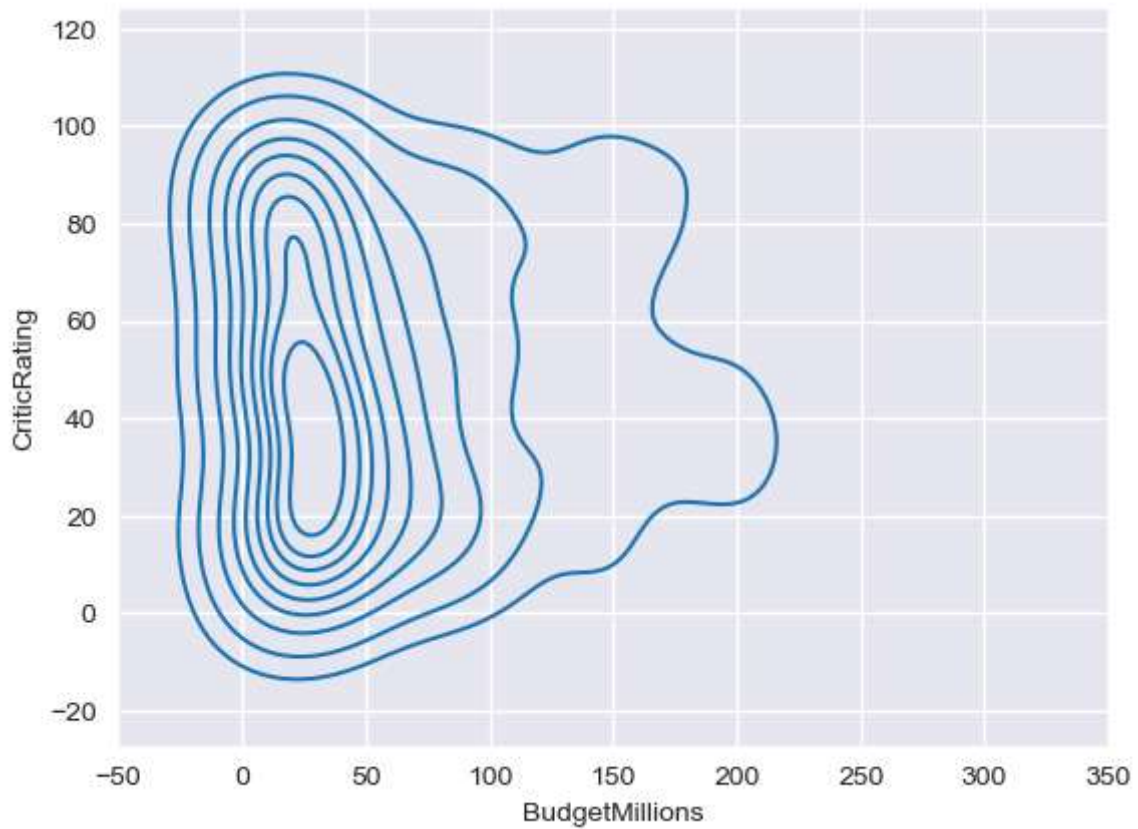


```
In [65]: k2 = sns.kdeplot(movies,x='CriticRating',y='BudgetMillions')
```



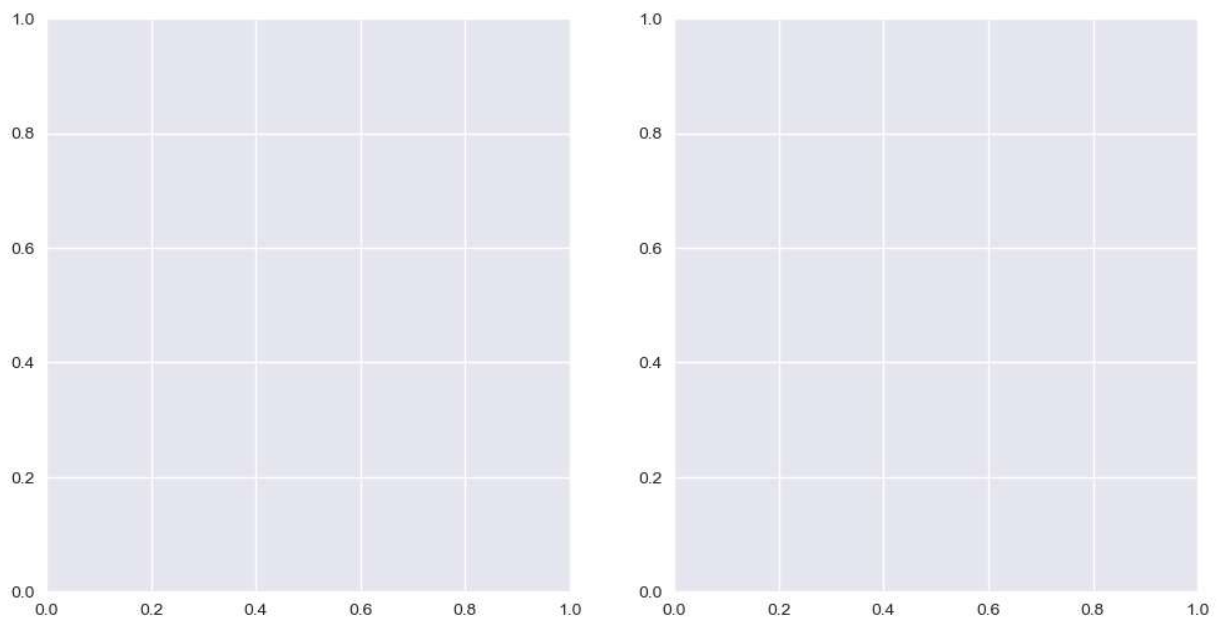
```
In [66]: k2 = sns.kdeplot(movies,x='BudgetMillions',y='CriticRating')
```





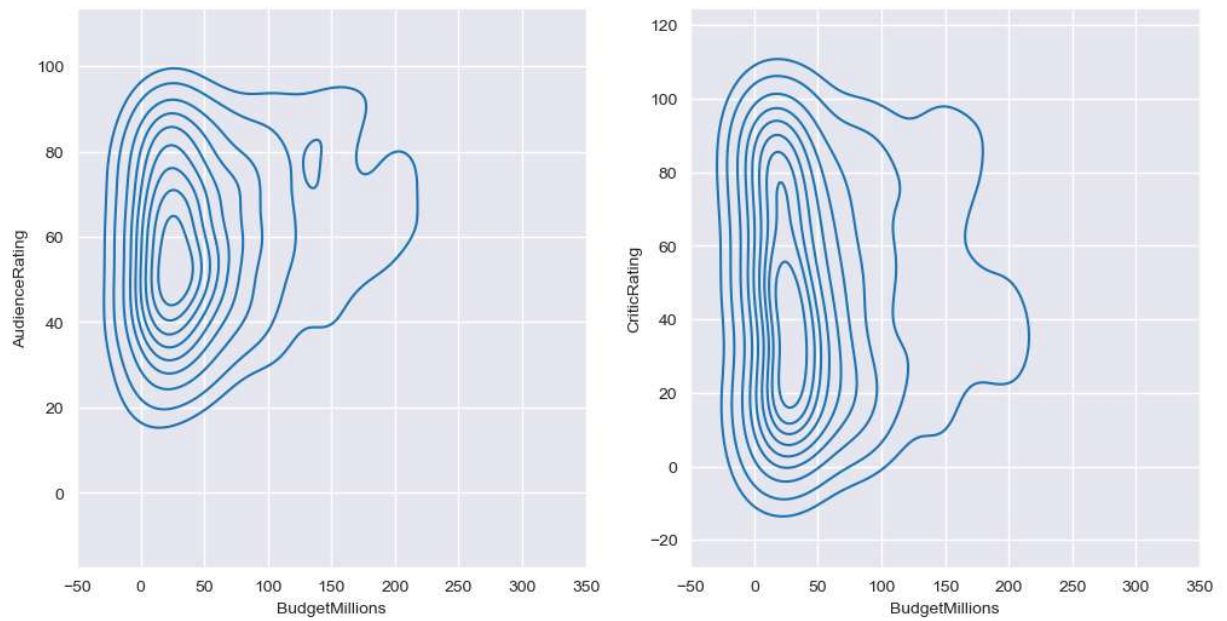
In [67]: `#subplots`

```
f, ax = plt.subplots(1,2, figsize =(12,6))
#f, ax = plt.subplots(3,3, figsize =(12,6))
```



In [68]: `f, axes = plt.subplots(1,2, figsize =(12,6))`

```
k1 = sns.kdeplot(movies,x='BudgetMillions',y='AudienceRating',ax=axes[0])
k2 = sns.kdeplot(movies,x='BudgetMillions',y='CriticRating',ax = axes[1])
```

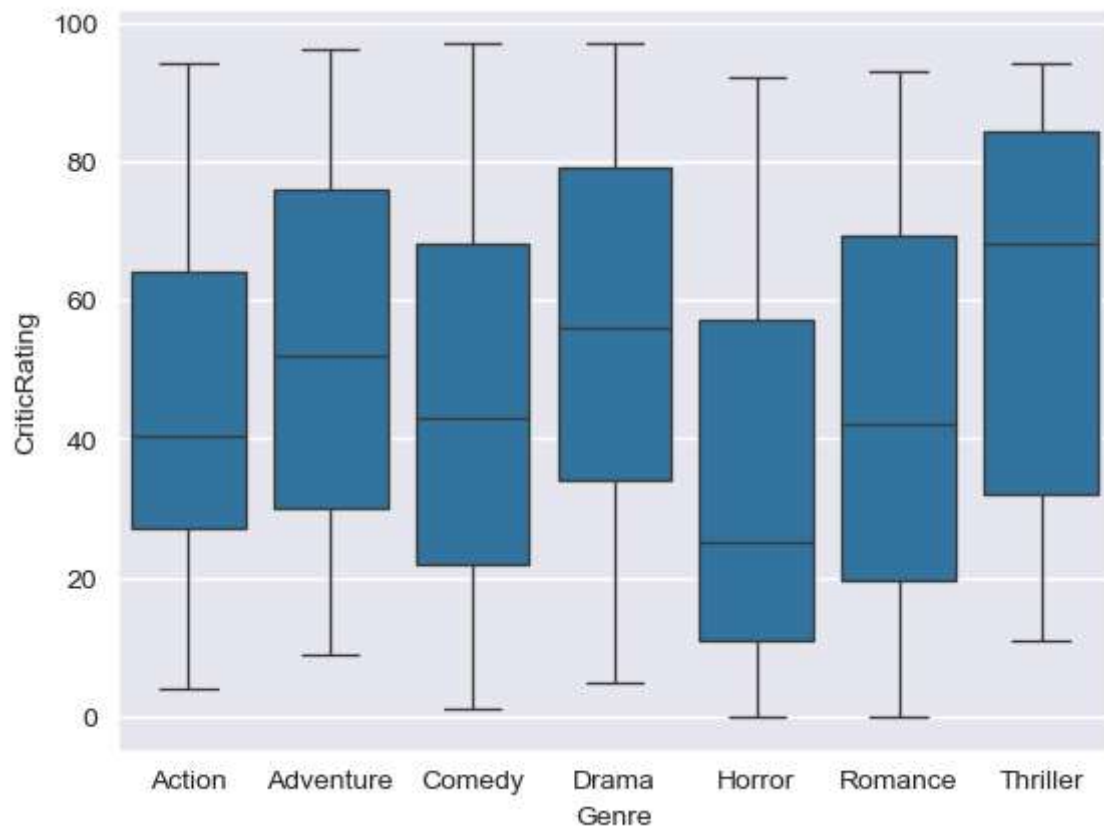


```
In [69]: axes
```

```
Out[69]: array([<Axes: xlabel='BudgetMillions', ylabel='AudienceRating'>,  
                <Axes: xlabel='BudgetMillions', ylabel='CriticRating'>],  
              dtype=object)
```

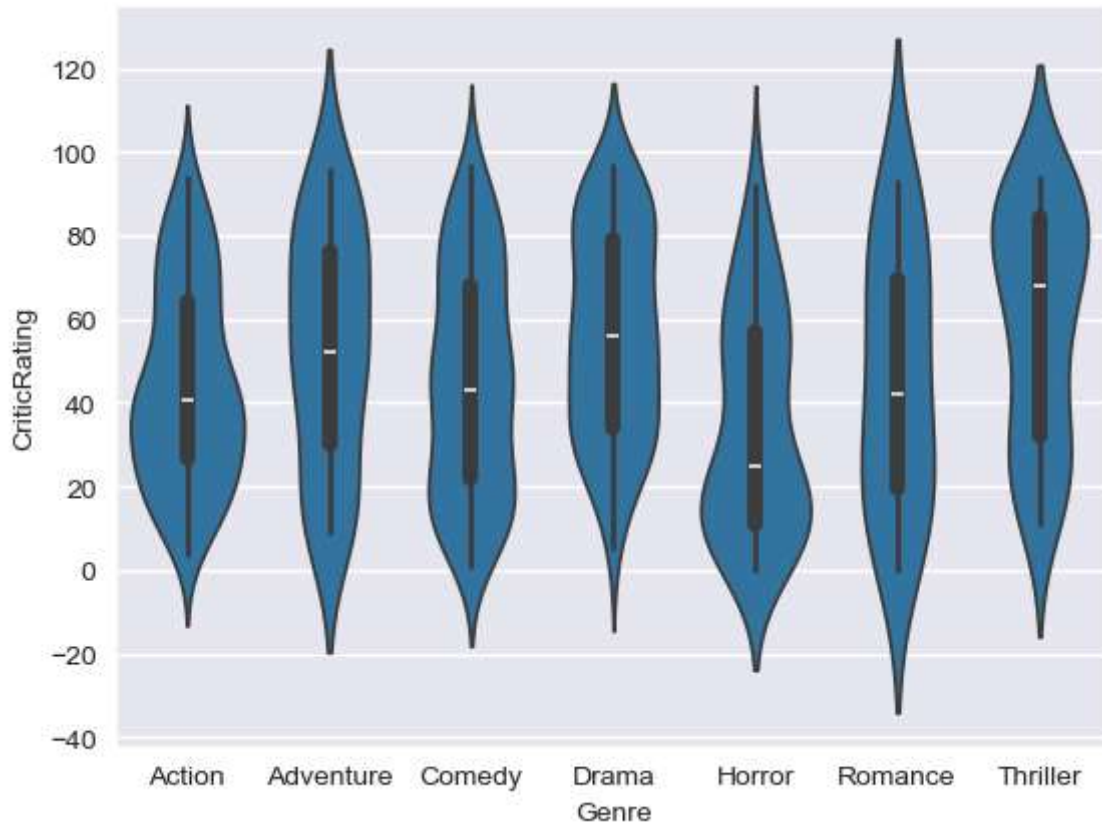
```
In [70]: #Box plots -
```

```
w = sns.boxplot(data=movies, x='Genre', y = 'CriticRating')
```

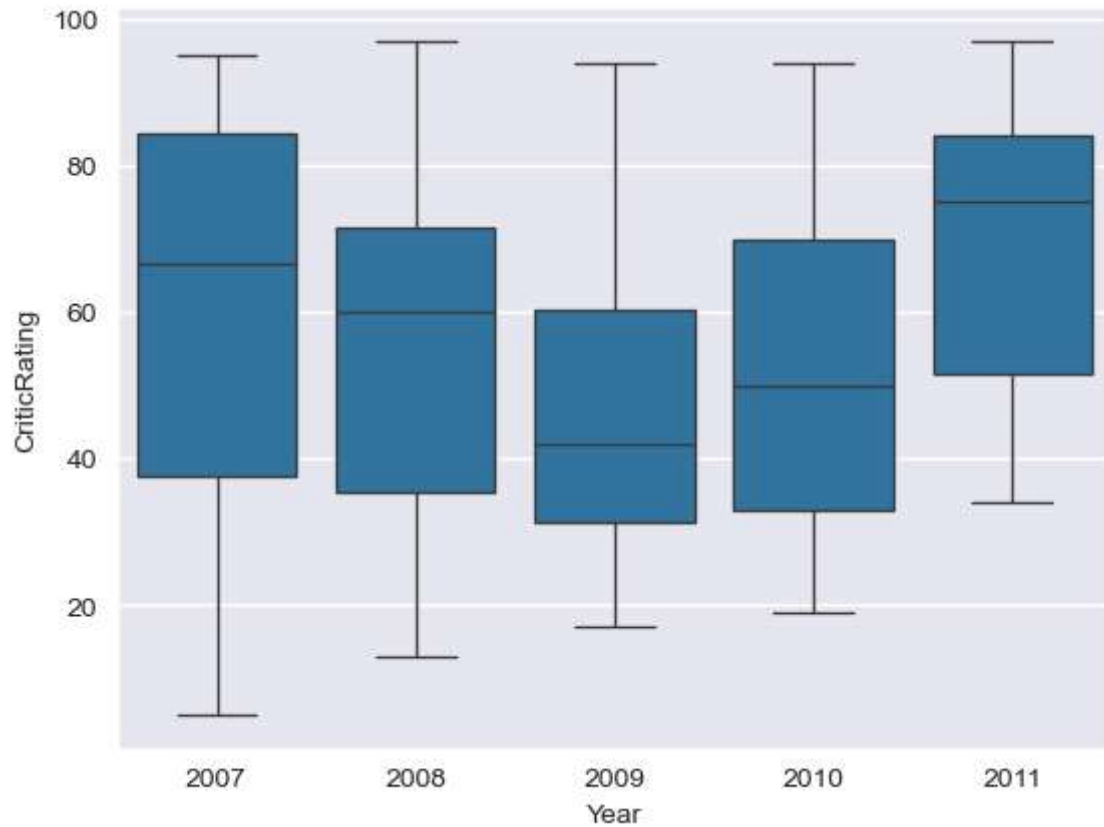


```
In [71]: #violin plot
```

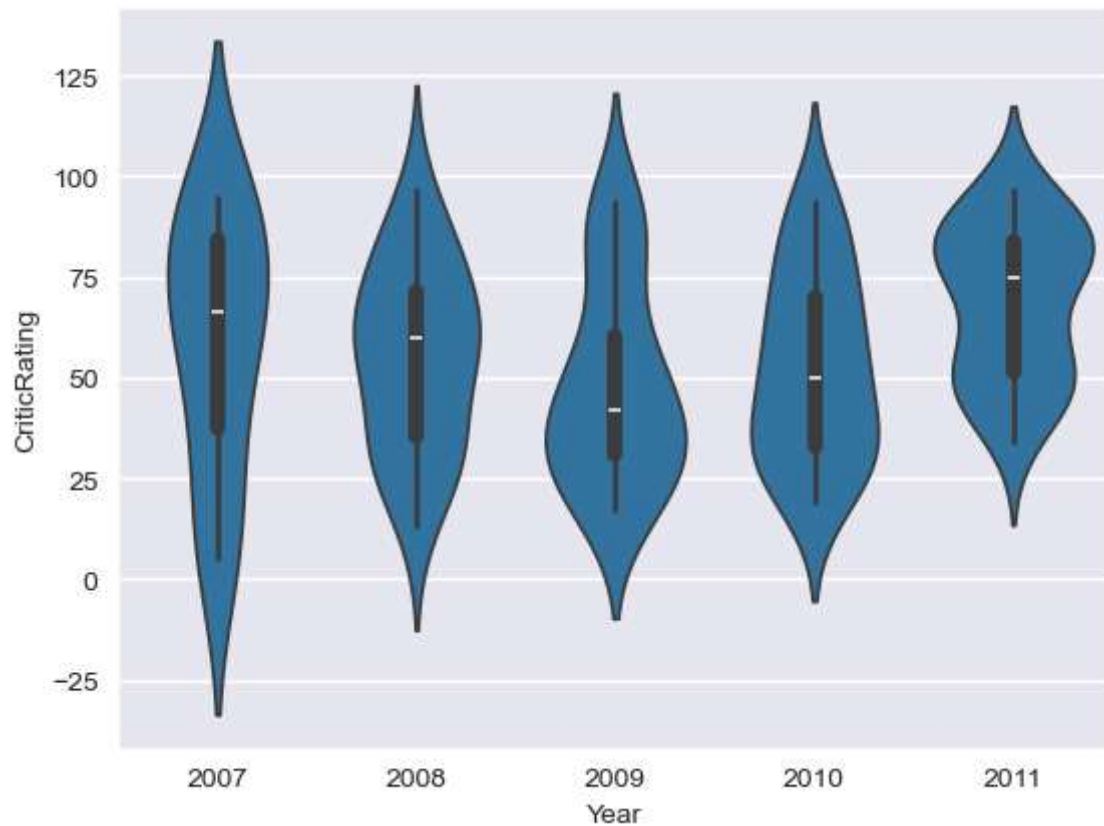
```
z = sns.violinplot(data=movies, x='Genre', y = 'CriticRating')
```



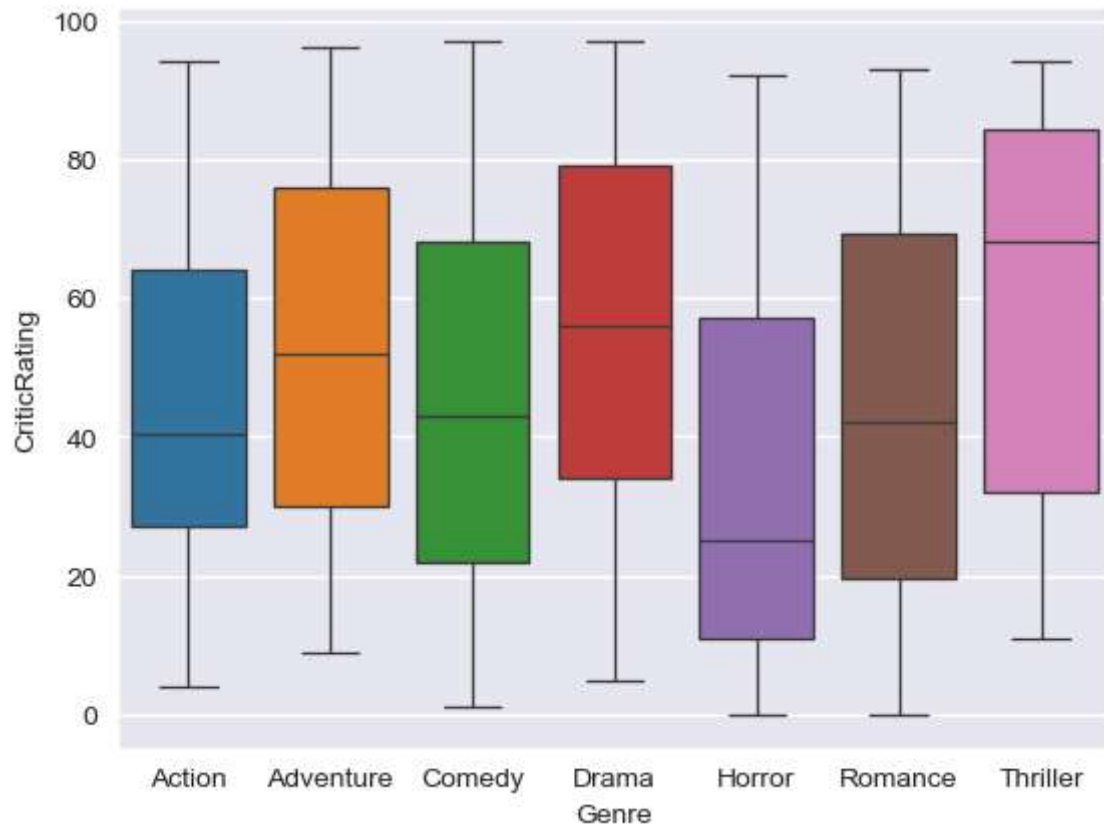
```
In [72]: w1 = sns.boxplot(data=movies[movies.Genre == 'Drama'], x='Year', y = 'CriticRating')
```



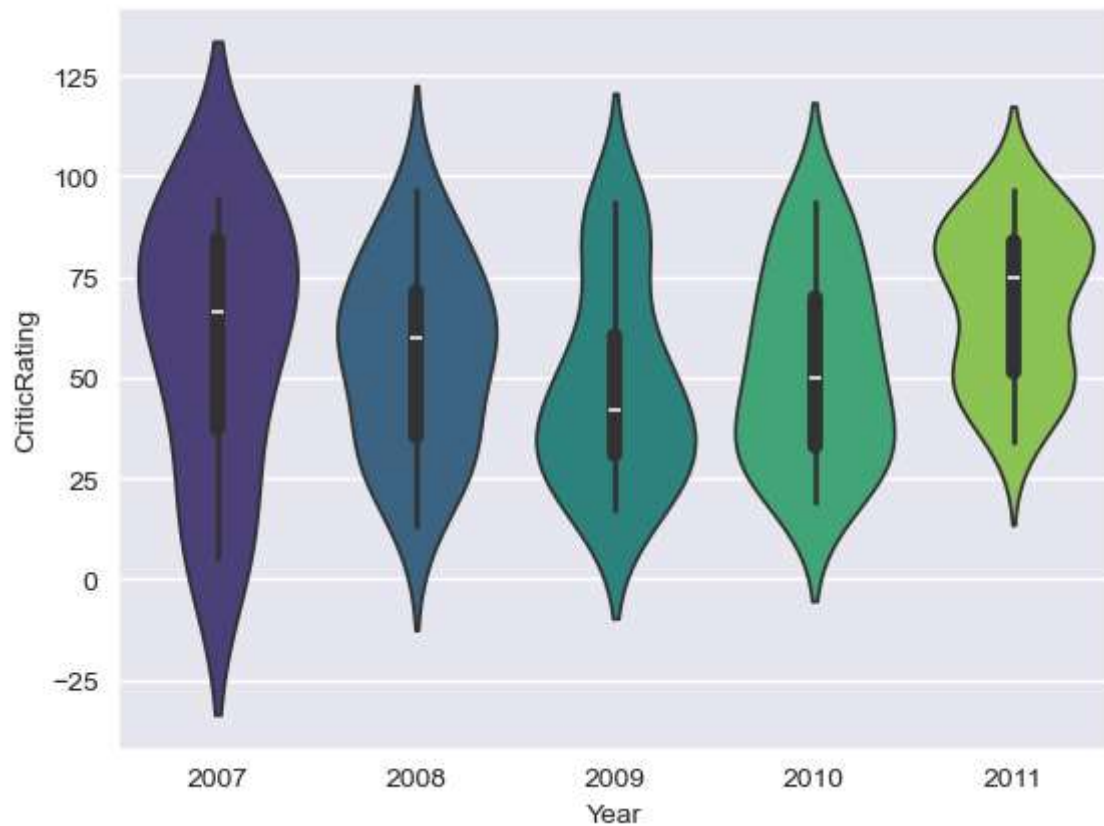
```
In [73]: z = sns.violinplot(data=movies[movies.Genre == 'Drama'], x='Year', y = 'CriticRating')
```



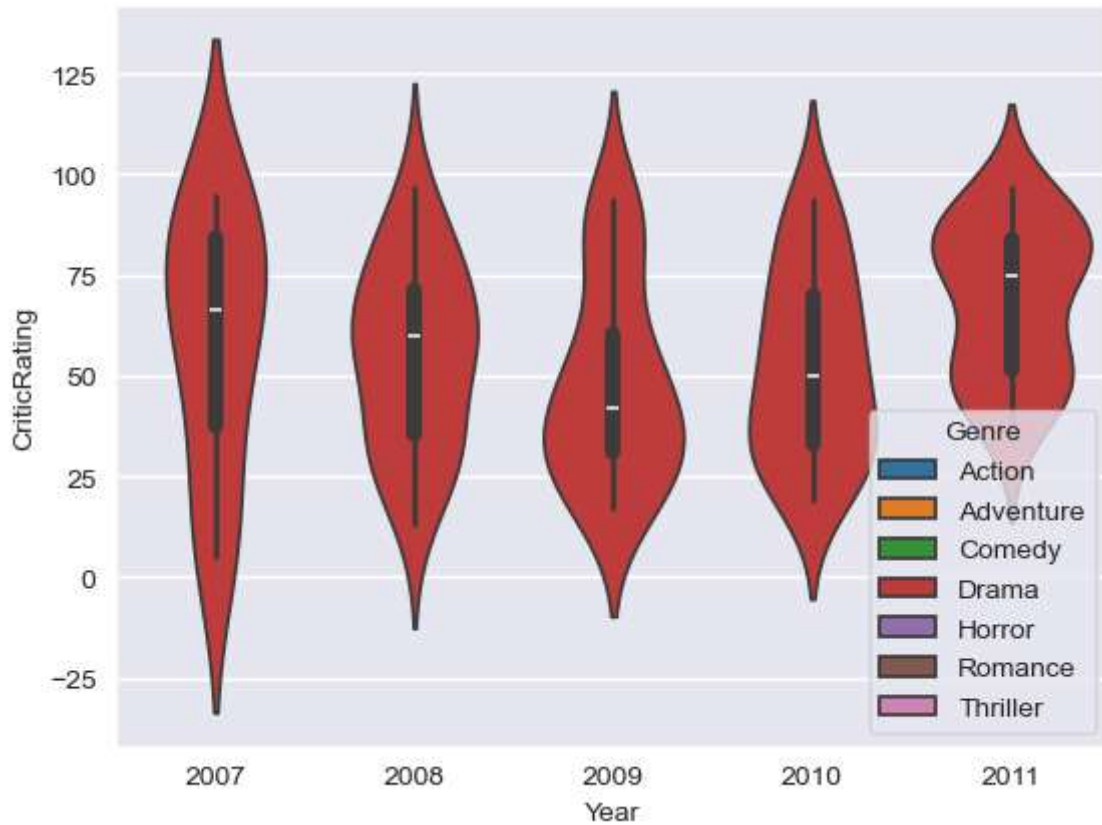
```
In [74]: z = sns.boxplot(data=movies, x='Genre', y = 'CriticRating', hue='Genre')
```



```
In [75]: z = sns.violinplot(data=movies[movies.Genre == 'Drama'], x='Year', y = 'CriticRating')
```

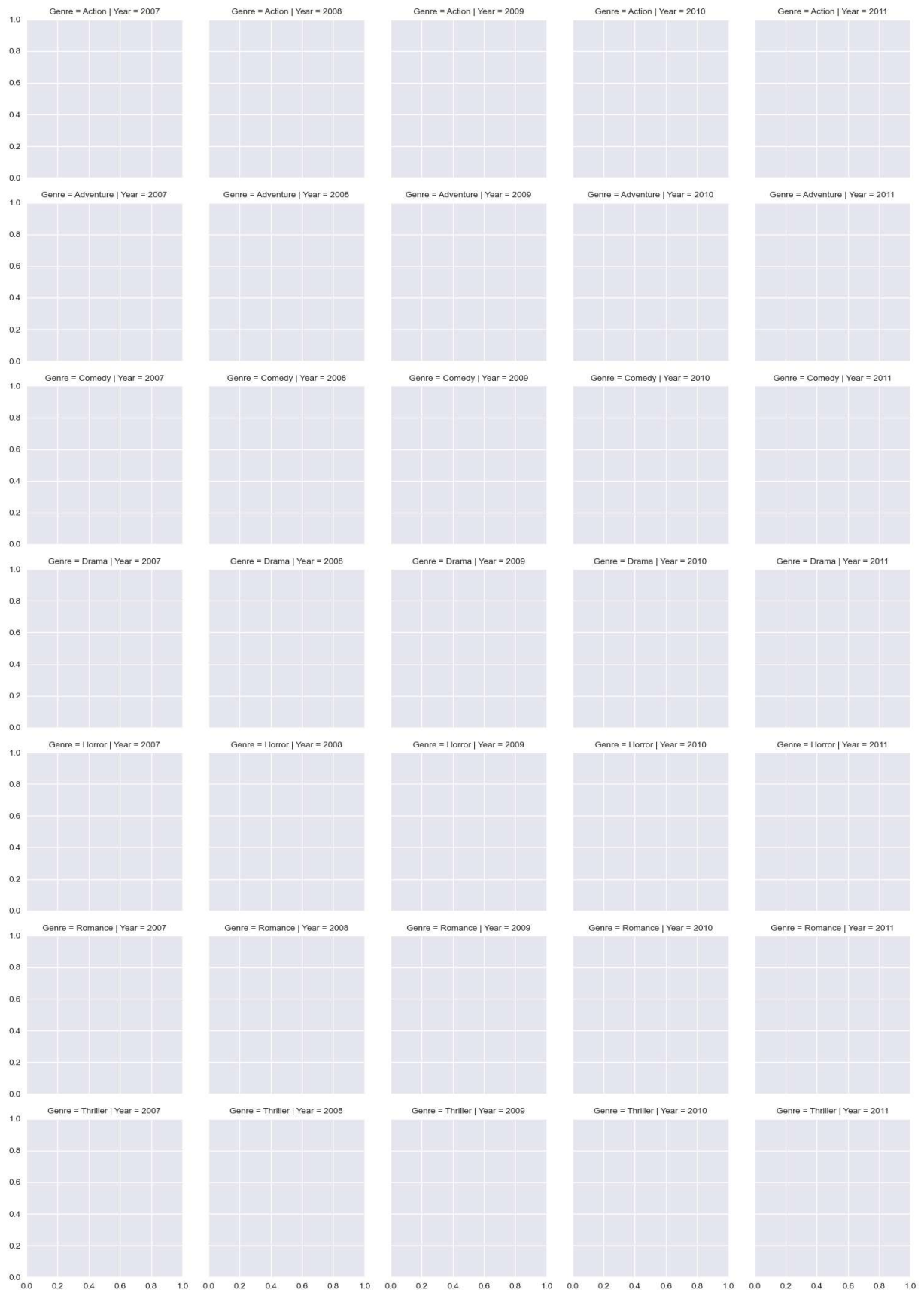


```
In [76]: z = sns.violinplot(data=movies[movies.Genre == 'Drama'], x='Year', y = 'CriticRating')
```



## creating a facet grid

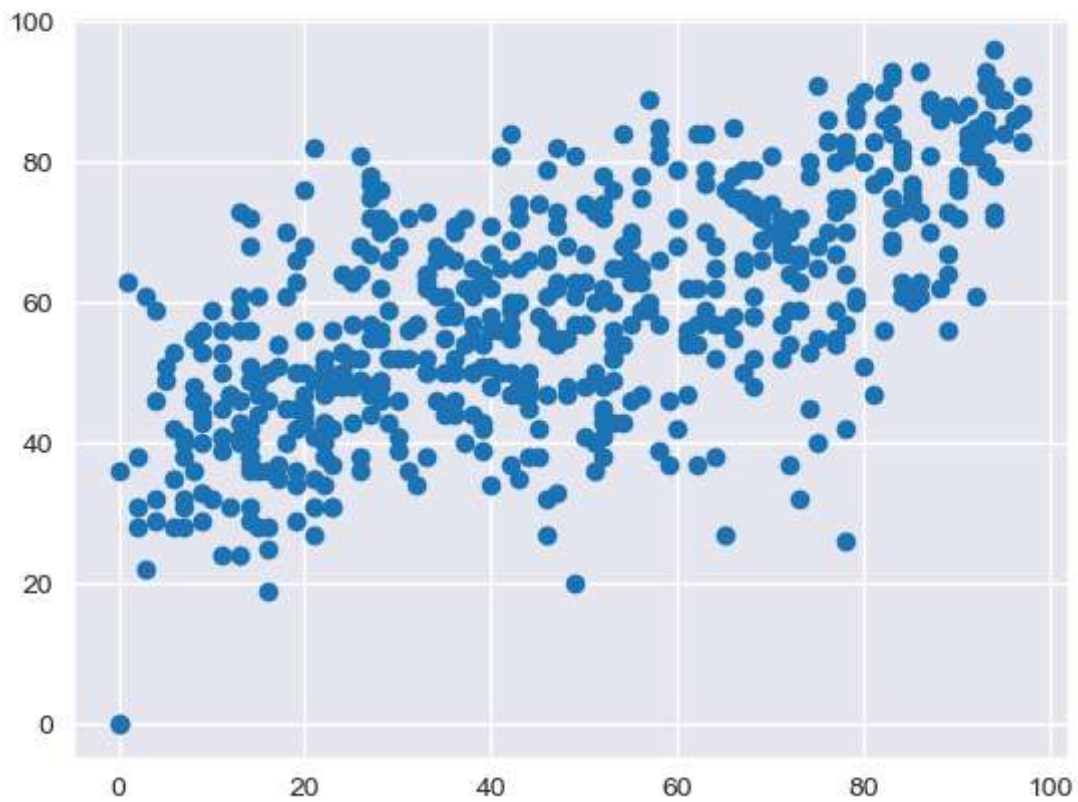
```
In [78]: g = sns.FacetGrid (movies, row = 'Genre', col = 'Year', hue = 'Genre') #kind of subp
```



```
In [79]: plt.scatter(movies.CriticRating,movies.AudienceRating)
```

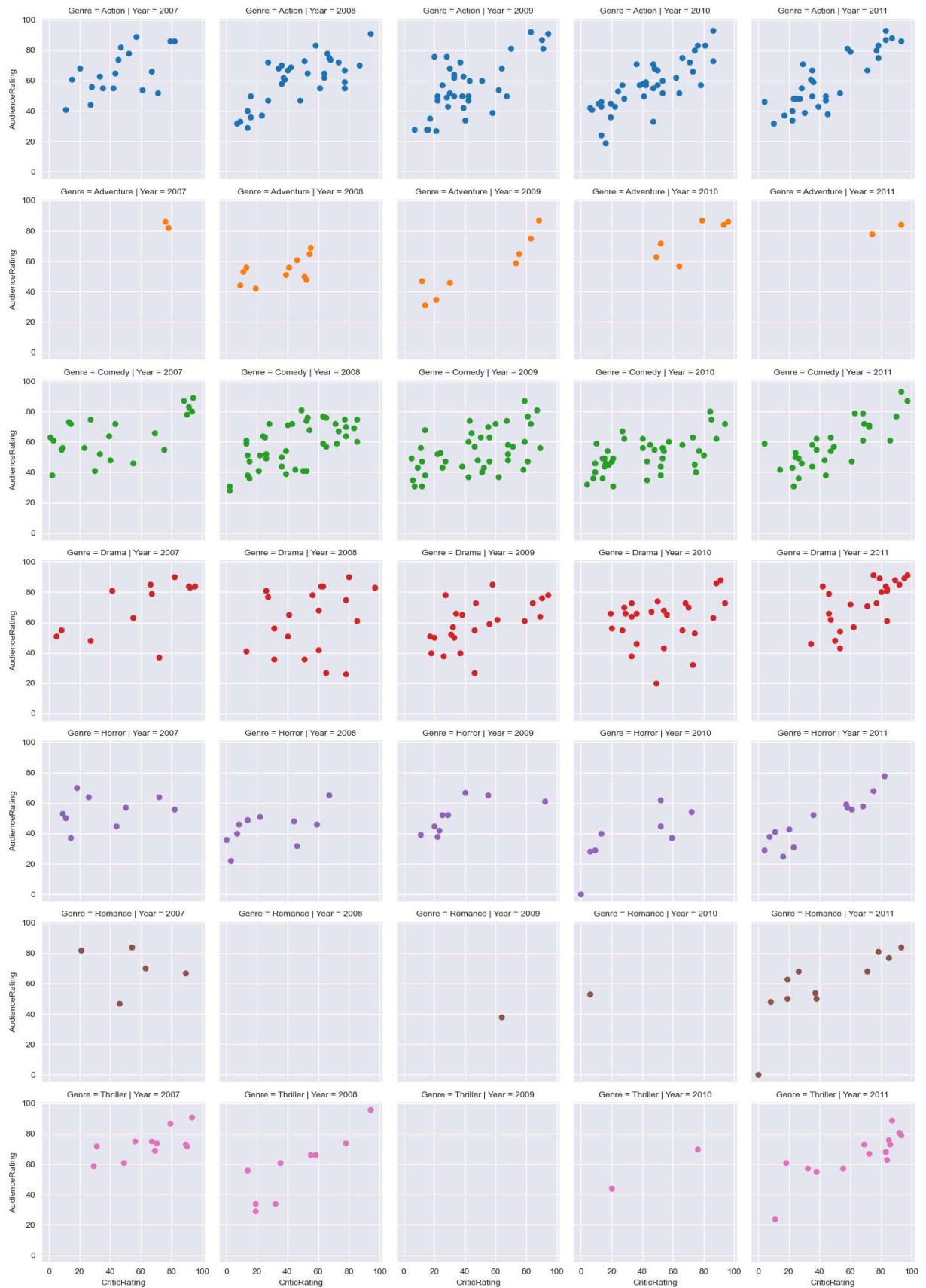
```
Out[79]: <matplotlib.collections.PathCollection at 0x16d225b9450>
```





```
In [80]: g = sns.FacetGrid (movies, row = 'Genre', col = 'Year', hue = 'Genre')  
g = g.map(plt.scatter, 'CriticRating', 'AudienceRating' ) #scatterplots are mapped
```



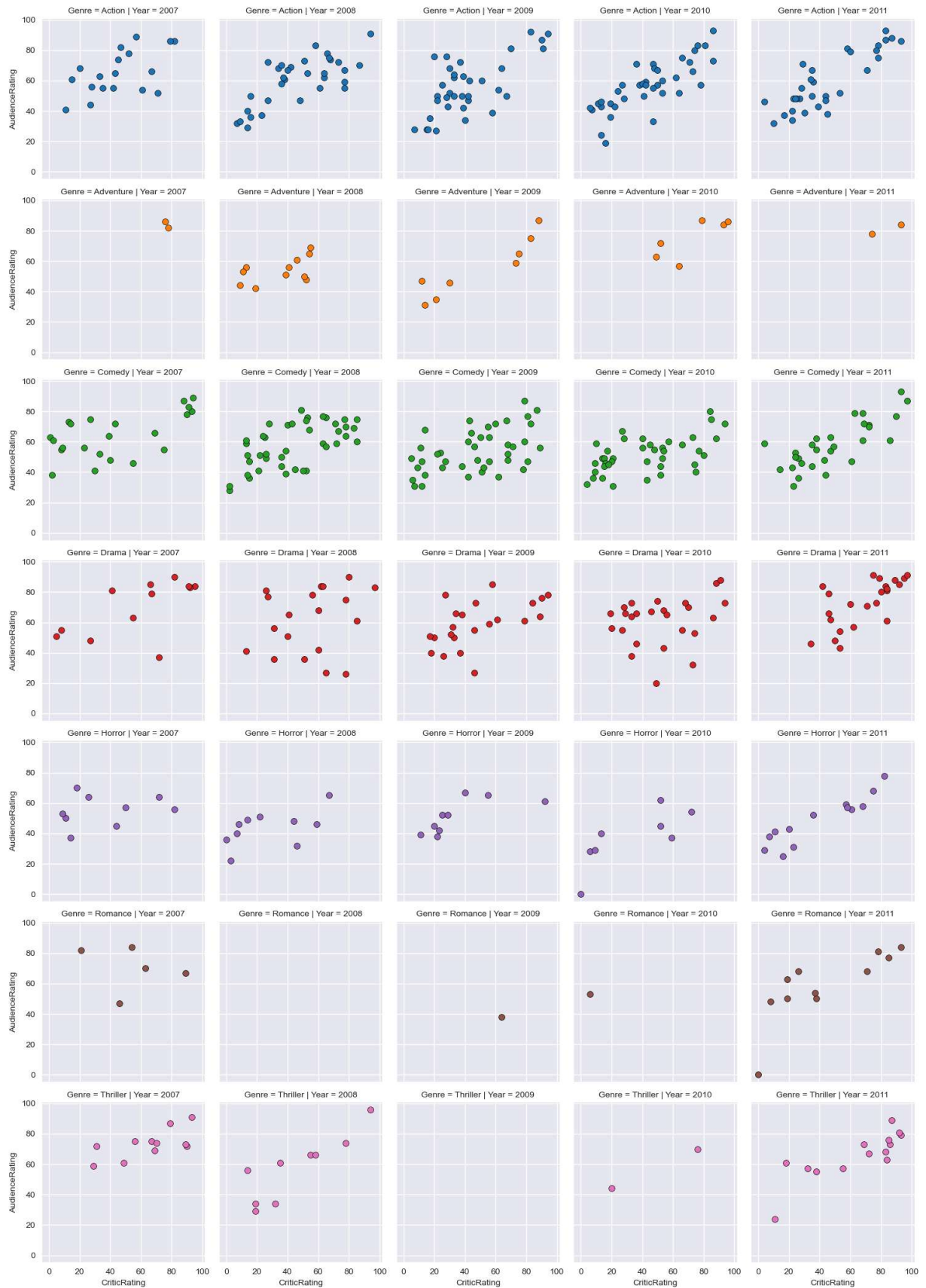


In [81]: *# you can populated any type of chat.*

```
g = sns.FacetGrid (movies, row = 'Genre', col = 'Year', hue = 'Genre')
g = g.map(plt.hist, 'BudgetMillions') #scatterplots are mapped in facetgrid
```

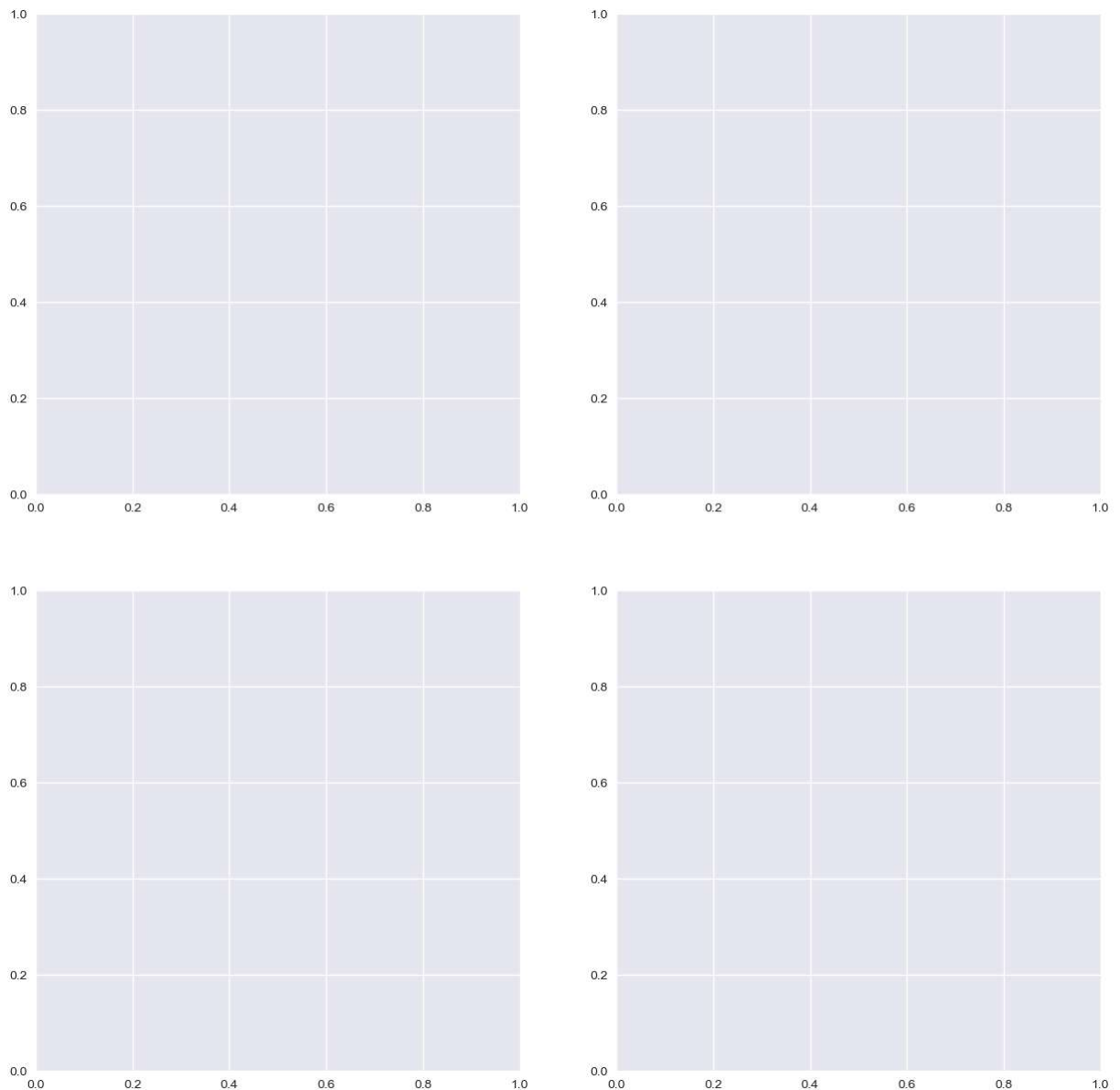


```
In [82]: #
g = sns.FacetGrid (movies, row = 'Genre', col = 'Year', hue = 'Genre')
kws = dict(s=50, linewidth=0.5, edgecolor='black')
g = g.map(plt.scatter, 'CriticRating', 'AudienceRating', **kws ) #scatterplots are m
```



```
In [83]: # python is not vectorize programming language
         # Building dashboards (dashboard - combination of chats)
```

```
sns.set_style('darkgrid')
f, axes = plt.subplots (2,2, figsize = (15,15))
```



```
In [87]: # python is not vectorize programming language
# Building dashboards (dashboard - combination of chats)

sns.set_style('darkgrid')
f, axes = plt.subplots (2,2, figsize = (15,15))

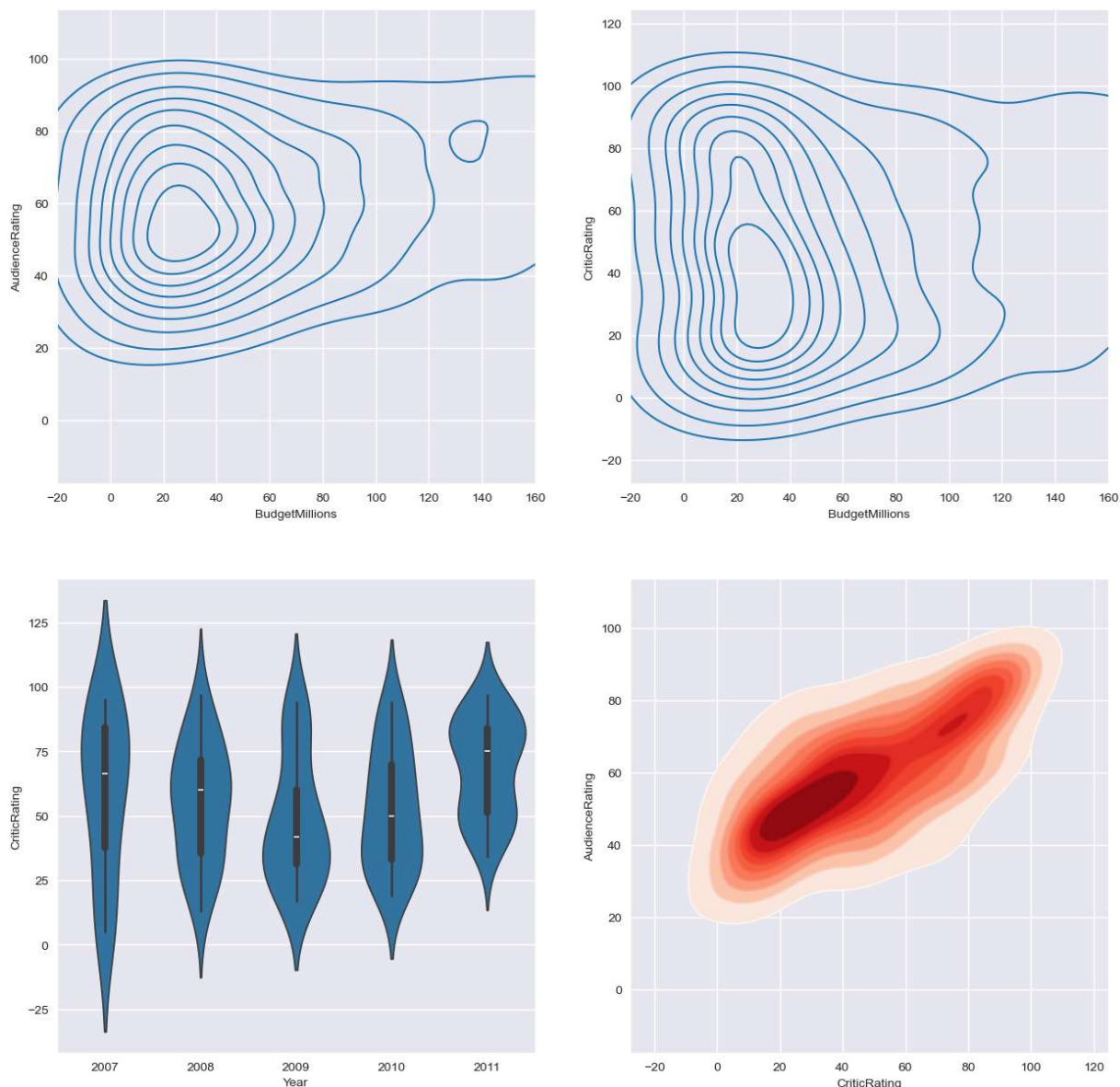
k1 = sns.kdeplot(movies,x='BudgetMillions',y='AudienceRating',ax=axes[0,0])
k2 = sns.kdeplot(movies,x='BudgetMillions',y='CriticRating',ax = axes[0,1])

k1.set(xlim=(-20,160))
k2.set(xlim=(-20,160))

z = sns.violinplot(data=movies[movies.Genre=='Drama'], x='Year', y = 'CriticRating')

k4 = sns.kdeplot(movies,x='CriticRating',y='AudienceRating',shade = True,shade_lowe
k4b = sns.kdeplot(movies,x='CriticRating',y='AudienceRating',cmap='Reds',ax = axes[
```

```
plt.show()
```



In [89]: *# How can you style your dashboard using different color map*

```
# python is not vectorize programming language
# Building dashboards (dashboard - combination of chats)

sns.set_style('dark',{'axes.facecolor':'black'})
f, axes = plt.subplots (2,2, figsize = (15,15))

#plot [0,0]
k1 = sns.kdeplot(data=movies,x='CriticRating',y='AudienceRating', \
                 shade = True,cmap = 'inferno', \
                 ax = axes[0,0])
k1b = sns.kdeplot(data=movies,x='CriticRating',y='AudienceRating', \
                  cmap = 'cool',ax = axes[0,0])

#plot [0,1]
k2 = sns.kdeplot(data=movies,x='CriticRating',y='AudienceRating',\
```

```
        shade=True, shade_lowest=True, cmap='inferno', \
        ax = axes[0,1])
k2b = sns.kdeplot(data=movies,x='CriticRating',y='AudienceRating', \
        cmap = 'cool', ax = axes[0,1])

#plot[1,0]
z = sns.violinplot(data=movies[movies.Genre=='Drama'], \
        x='Year', y = 'CriticRating', ax=axes[1,0])

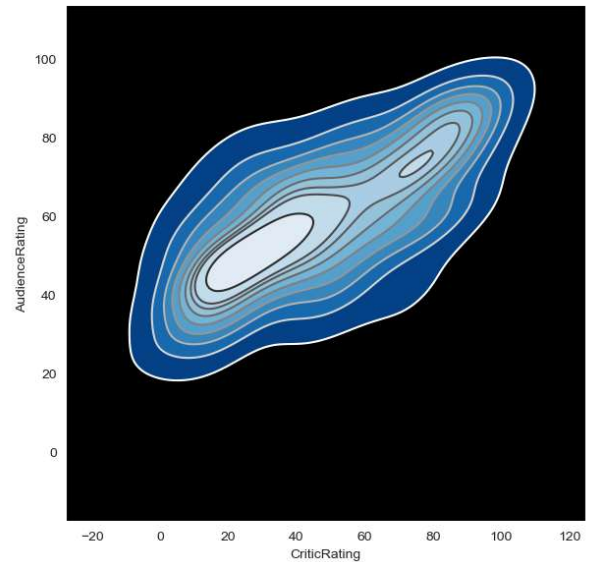
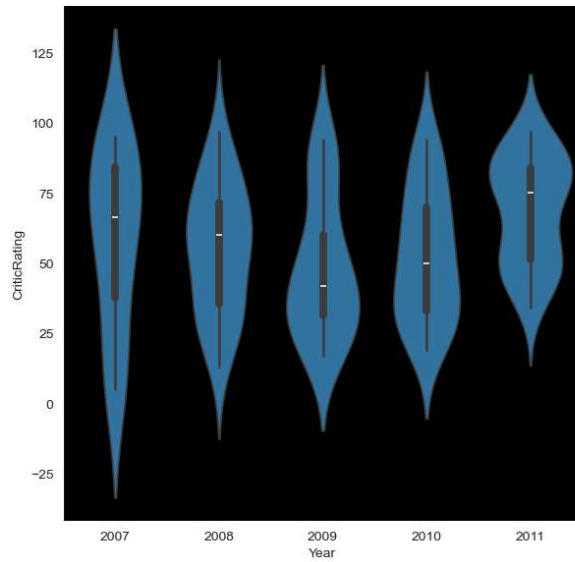
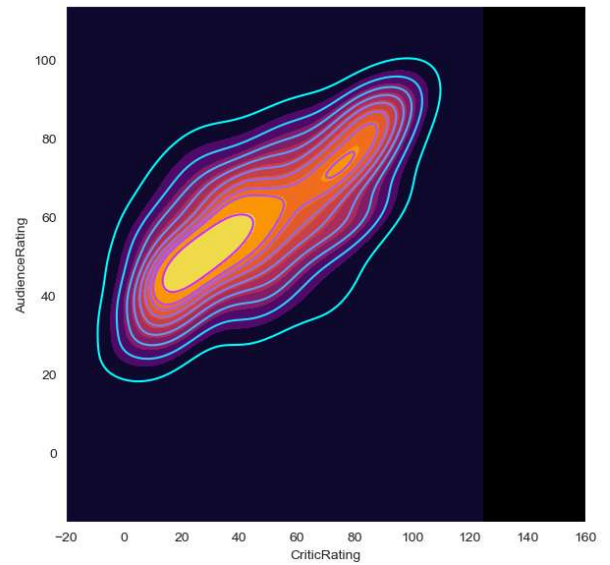
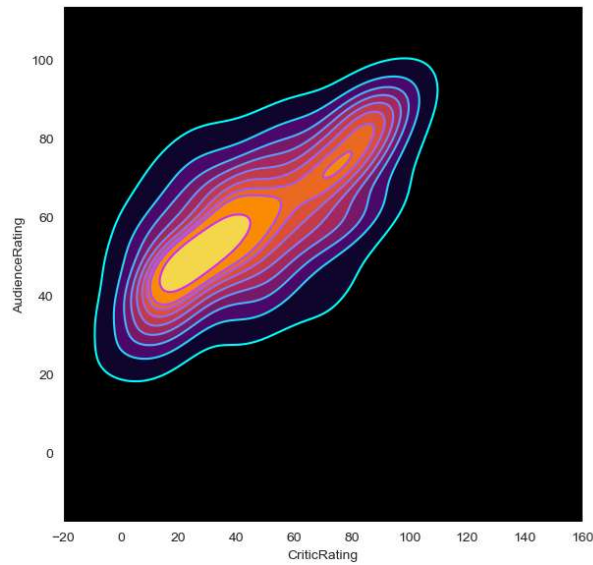
#plot[1,1]
k4 = sns.kdeplot(data=movies,x='CriticRating',y='AudienceRating', \
        shade = True,shade_lowest=False,cmap='Blues_r', \
        ax=axes[1,1])

k4b = sns.kdeplot(data=movies,x='CriticRating',y='AudienceRating', \
        cmap='gist_gray_r',ax = axes[1,1])

k1.set(xlim=(-20,160))
k2.set(xlim=(-20,160))

plt.show()
```





In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:



In [ ]:

In [ ]: